Smart Aqua

Fish AI Detection and Clustering Dissertation

Project focus: automated fish detection, crop extraction, quality filtering, unsupervised clustering, and dashboard-based workflow support

Student name: David Wilson
Student ID: 10781607
Course: Comp3000
Supervisor: Vassilis Cutsuridis
Submission date: 05/05/2026

Git Link: https://github.com/DavidWilson2000/COMP3000
Video Link: https://youtu.be/v2P2-3l6aR0

Word Count: 10,932

Abstract

Smart Aqua is a lightweight fish video analysis system designed to support the detection, extraction, filtering, and organisation of fish from aquarium footage. The project was developed in response to the challenge of manually reviewing fish video data, which can be time-consuming and difficult when footage contains cluttered backgrounds, overlapping fish, varying lighting conditions, and motion blur. The system uses a custom-trained YOLOv8 detector to identify fish in video frames and generate bounding boxes for crop extraction. These crops are then passed through a quality filtering stage to remove unsuitable outputs before being grouped using ResNet50 feature extraction and HDBSCAN clustering. A Tkinter-based dashboard was also created to make the workflow easier to run, inspect, and demonstrate.

The final detector achieved a precision of 0.882, recall of 0.793, mAP50 of 0.859, and mAP50-95 of 0.657, showing that the model was effective at detecting fish while still leaving room for improvement in recall and stricter bounding box accuracy. The final system does not claim to provide confirmed species classification; instead, the clustering stage provides exploratory visual groupings to support manual review. Overall, Smart Aqua demonstrates that an accessible AI-assisted workflow can be built using local hardware and available tools, providing a practical foundation for future fish monitoring and analysis systems.

Table of Contents

1. Introduction

1.1 Project Context and Problem Statement

Fish maintenance and management have been a long-standing challenge for farmers and researchers, particularly because technical advances have been adopted more slowly in this field than in others. Although AI has existed for decades and has become increasingly mainstream in recent years (Tableau, n.d.), its adoption within marine and fisheries settings has remained comparatively limited. In cases where it has been applied, it has often been developed for larger-scale operations, which can make such systems difficult for smaller fishery practitioners and farmers to access, maintain, and train. Marine AI has increasingly been explored in aquaculture for tasks such as feeding efficiency, biomass estimation, growth tracking, disease detection, environmental monitoring, and labour reduction (Lutz, 2023). However, these systems can still be expensive to develop, maintain, and train, making them less accessible for smaller practitioners. Smart Aqua contributes to a lightweight end-to-end workflow. It combines fish detection, crop extraction, filtering based on quality, clustering based on visual similarities and a user-facing dashboard. It is designed for practical use by a wider range of users rather than as a fully specialist, resource-heavy tool.

1.2 Aims and User Benefit

Smart Aqua aims to create a lightweight and easy-to-train YOLO-based detector that allows researchers and farmers to observe different fish with greater accuracy and f**ocus on relevant fish groups or retrain the detector for a target fish type where suitable data is available** without time-consuming manual observation, which may disrupt the ecosystem. It allows users to easily process the data via an easy-to-understand dashboard interface regardless of technical skill and see relevant results in a timely manner. The processed video can then be used to review all detected fish, or only the relevant cluster, and check for abnormalities.

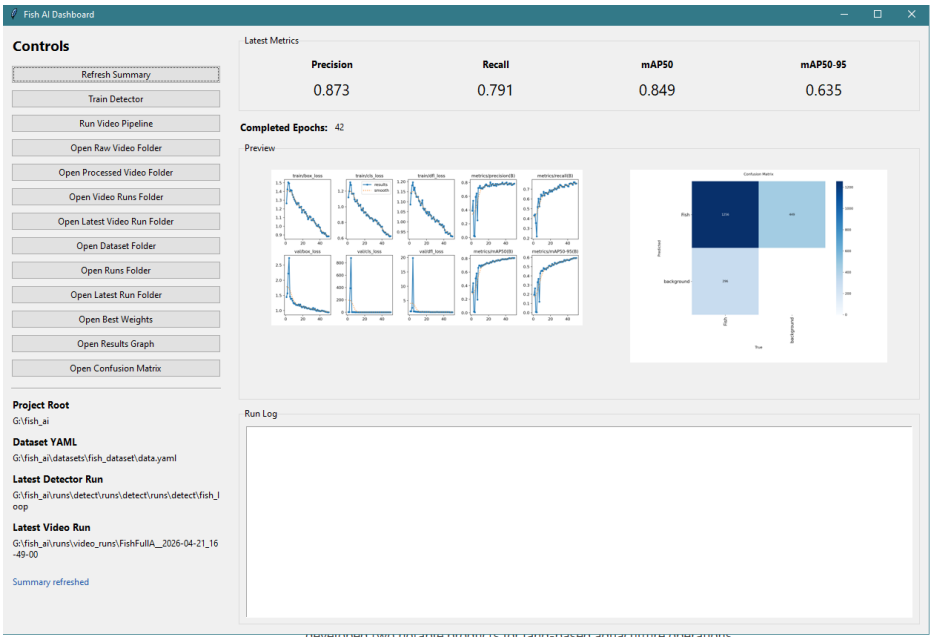


Figure 1: Dashboard interface used to launch pipeline stages, access project files, and review model outputs.

1.3 Objectives

Develop a robust fish detector trained on a custom dataset.
Build an end-to-end video analysis pipeline from frame extraction to clustered crop output.
Evaluate detector performance using precision, recall, mAP50, and mAP50-95.
Assess the effectiveness of downstream crop filtering and clustering.
Develop a lightweight dashboard interface to support repeatable use and demonstration.

1.4 Research Questions

- How effectively can a custom YOLO-based detector identify fish in challenging video frames?
- How do dataset quality and pipeline design choices affect downstream crop quality and cluster effectiveness?
- To what extent does the final system provide a practical workflow for fish video analysis?

1.5 Scope

Smart Aqua combines a trained single-class fish detection model with an unsupervised clustering stage. The detection model is used to identify fish in video frames and generate bounding boxes around them. The clustering stage does not train a separate species classifier; instead, it uses a pretrained ResNet50 model to extract visual features from cropped fish images, then applies HDBSCAN to group visually similar crops into unnamed clusters. This design choice was made because training a fully supervised species classifier would have required reliable species-labelled data and would have increased the complexity of the system. As a result, the clusters should be treated as exploratory visual groupings rather than authoritative species classifications.

The project also includes an executable dashboard interface that allows users to operate the pipeline and review outputs without requiring direct interaction with the underlying scripts. This was done to help keep the system accessible to non-specialist users rather than targeting full enterprise deployment.

2. Background, Domain Context, and Related Work

2.1 Computer Vision for Object Detection

“Object detection is a technique that uses neural networks to localise and classify objects in images” (IBM, n.d.). This means the model can assign class labels to relevant objects within the scene. It does this by being trained on a large amount of data with bounding boxes drawn and training against that data to refine predictions. The model performs object localisation by creating a bounding box around the detected object, after it checks against a confidence threshold to see if the detection is accepted. This allows fish to be found before cropping, filtering, and clustering. Unlike image classification, which assigns a label to a whole image, object detection identifies both the class and location of objects within the image. This was important for Smart Aqua because the later crop extraction stage depends on the detector producing usable bounding boxes. This can cause challenges when there is a crowded scene of overlapping or partially obscured fish of varying sizes. This means that high-quality training data is needed to ensure accurate detection.

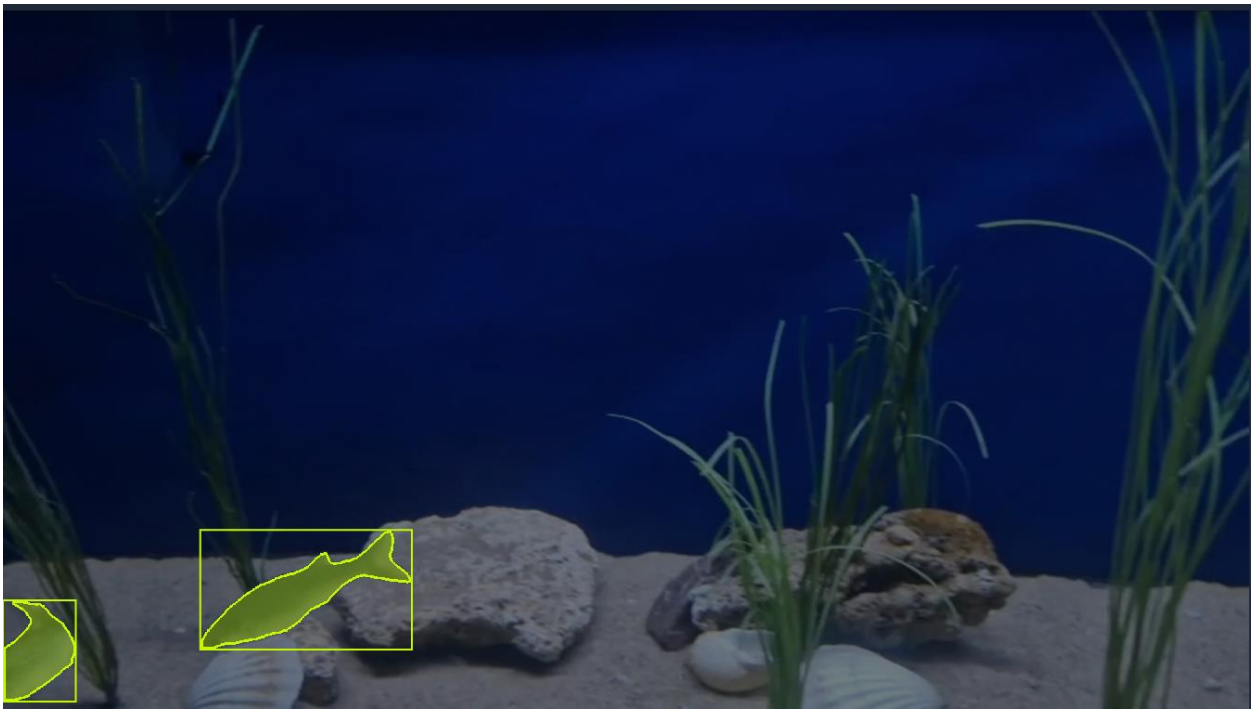


Figure 2: An example annotated training image

2.2 YOLO-Based Detection

YOLO uses an end-to-end neural network to predict bounding boxes and class probabilities in one pass over an image (Kundu, 2023). This makes it well suited to object detection such as Smart Aqua where speed and localisation accuracy are key. Unlike two-stage detectors, which first generate candidate regions before classifying them, YOLO performs detection as a single-stage process. This allows the image to be processed in one forward pass, making it suitable for workflows where speed is important. This was helpful for Smart Aqua because the system needs to analyse video frames, then detect fish before passing the resulting bounding boxes into later crop extraction

YOLOv8 was selected because it provides a strong balance between detection accuracy, speed, and practical usability, and has been applied in research contexts where object detection is required from image or video data (Hermens, 2024). Ultralytics describes YOLOv8 as using improved backbone and neck architectures, an anchor-free detection head, and a range of pretrained model sizes, allowing developers to choose between lightweight and more accurate variants depending on the project requirements. For this project, these features were important because fish in underwater footage can appear at different sizes, positions, and levels of visibility. The availability of pretrained weights also made it possible to fine-tune the detector on a custom fish dataset rather than training a model from the beginning. In addition, the Ultralytics Python workflow supports training, validation, prediction, and export in a consistent format, making YOLOv8 easier to integrate into the wider Smart Aqua pipeline and dashboard

2.3 Dataset Quality in Detection Projects

Dataset quality is a major factor in object detection performance because the model learns both what the target object looks like and where it appears in an image. If training images are blurry, repetitive,

poorly labelled, or inconsistent, the detector may learn unreliable patterns and perform poorly on unseen footage.

This was an important issue for Smart Aqua because clear and varied fish footage was needed. The original intention was to collect data from the Plymouth Aquarium, but this was unavailable, so footage was gathered from online platforms and videos recorded on a mobile phone. Frames were then extracted and reviewed for suitability, with selection based on fish visibility, frame clarity, fish density, variation in appearance, size, colour, and lighting. This helped reduce bias and avoid training the detector only on one narrow type of footage.

Annotation quality was also important. The selected frames were manually labelled in Roboflow using bounding boxes, with each annotation assigned to the single class “Fish”. Consistent annotation helped the detector learn the relationship between fish features and bounding box locations, while reducing the risk of confusing training data caused by missed fish, inaccurate box placement, or inconsistent labelling.

2.4 Metrics

- Precision: proportion of predicted fish that are correct.
- Precision is needed to ensure that detections accepted by the model are genuine fish rather than background objects. This was important for Smart Aqua because false positives, such as reefs or other underwater objects, could lead to irrelevant crops being passed into the filtering and clustering stages.

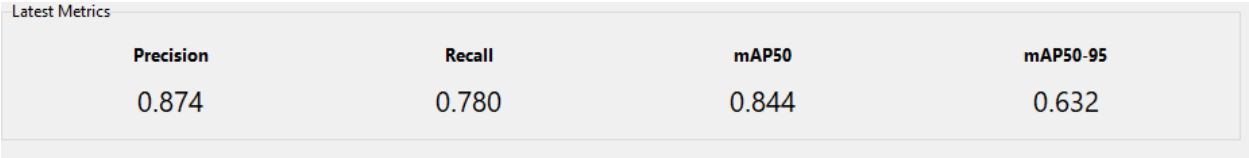
- Recall: proportion of true fish that are successfully detected.
  Recall is needed to measure whether the detector is finding as many true fish as possible in each frame. This is important for Smart Aqua because missed detections reduce the number of usable crops and limit the variety of data available for later clustering.

- mAP50: average precision at IoU 0.50.
  IoU, or Intersection over Union, measures how much the predicted bounding box overlaps with the manually labelled ground-truth bounding box. A higher IoU means the model has placed the box more accurately around the object.
- Mean Average Precision, or mAP, summarises detection performance by considering how accurately the model detects objects across different confidence levels.

- mAP50 measures how well the detector identifies fish when a predicted bounding box overlaps with the labelled box by at least 50%. This is useful because it gives an overall view of detection performance while allowing some tolerance in box placement. For Smart Aqua, mAP50 is important because the pipeline depends on the detector finding fish reliably before crops can be extracted for filtering and clustering.

- mAP50-95: average precision across stricter IoU thresholds.

- mAP50-95 is a stricter metric because it evaluates the detector across multiple overlapping thresholds, not just 0.50. This means it considers whether the model can place bounding boxes accurately as well as whether it can detect fish at all. For Smart Aqua, this is useful because poorly placed boxes could produce low-quality crops, which would then affect the later filtering and clustering stages.
-



-

Figure 3: Example Metrics obtained by the model being trained

These metrics were selected because they are commonly used in object detection evaluation. Ultralytics reports precision, recall, mAP50, and mAP50-95 as standard YOLO validation outputs, while COCO-style evaluation commonly uses average precision across IoU thresholds from 0.50 to 0.95 to provide a stricter view of localisation quality (Ultralytics, n.d.; Padilla, Netto and da Silva, 2021). This made the metrics suitable for Smart Aqua because the project needed to evaluate both whether fish were detected and whether the bounding boxes were accurate enough for crop extraction.

2.5 Unsupervised Grouping / Clustering

Clustering was used in Smart Aqua to organise cropped fish detections into groups based on visual similarity. This stage should not be treated as a final species classification system because the system does not use verified species labels for each cropped fish. Instead, clustering provides an exploratory way to review fish crops that look similar, which can support later manual review or species-focused analysis.

In the implemented pipeline, the detector first identifies fish and produces cropped images. These filtered crops are then passed into the clustering stage. A pretrained ResNet50 model is used as a feature extractor, meaning it converts each crop into a numerical embedding that represents its visual appearance rather than assigning a species label. The clustering process then uses HDBSCAN to group similar embeddings together. Crops that do not fit clearly into a group are placed into an unknown cluster. This was useful for Smart Aqua because it allowed the system to organise fish crops without requiring a fully labelled species dataset or training a separate supervised species classifier, which would have reduced the lightweight nature of the system.

The main benefit of this approach is that it keeps the system lightweight while still adding value after detection. Users can inspect grouped crops more easily than reviewing every crop individually, while still understanding that the clusters are only visual groupings and not authoritative species names.

2.6 User-Facing Workflow Tools

The dashboard was added to make the Smart Aqua pipeline easier to use and demonstrate. Although the interface is simple, it allows users to run the main stages of the pipeline without needing to open or edit the code files directly. This supports the aim of making the system more accessible to users who may not have strong technical knowledge.

The dashboard also improves the repeatability of the workflow because important folders, model outputs, metrics, and result images can be accessed from one place. This is useful for the dissertation because it makes the system easier to test, review, and demonstrate. Instead of the project only existing as separate Python scripts, the dashboard helps present Smart Aqua as a more complete workflow tool. This also supports reproducibility because the same workflow can be launched through consistent controls rather than relying on manually running scripts in the correct order.

2.7 Critical Positioning of This Project

Smart Aqua is positioned as a lightweight end-to-end fish video analysis workflow rather than a single isolated AI model. The project combines several stages, including frame extraction, fish detection, crop extraction, crop quality filtering, visual clustering, and a dashboard interface. Each stage supports the next, meaning the value of the system comes from the full pipeline rather than from one script alone.

This is important because fish detection on its own would only show where fish appear in an image. In Smart Aqua, the detected fish are passed into later stages so that usable crops can be extracted, poor-quality crops can be reduced, and visually similar fish can be grouped for review. The dashboard then makes these outputs easier to access and understand. This makes the project stronger as a practical system because it shows how object detection can be used as part of a wider workflow for fish video analysis.

3. Requirements and System Design

3.1 Functional Requirements

Functional Requirement	Purpose
Import raw video files.	Allow users to provide videos for the pipeline
Extract frames at a configurable interval.	Convert video into frames to be used later in the pipeline
Select more informative frames for downstream processing.	Used to ensure the frames that make it to the final part of the pipeline are usable
Detect fish and save cropped detections.	Used to produce the main AI output for later processing
Filter poor-quality or duplicate crops.	Reduce noisy outputs before clustering and improve the usefulness of the final grouped crops.
Cluster remaining crops into visually similar groups.	Allow users to manually review all the relevant crops
Expose relevant folders, metrics, and outputs through a simple dashboard.	Allow any user to interact with the model without needing technical skills.

3.2 Non-Functional Requirements

Repeatable workflow: The same input video and configuration should produce a consistent folder structure and comparable outputs.

Reasonable processing time: The system should be able to run on the available project hardware without requiring enterprise-level computing resources.

Interpretable outputs: The system should save visible crops, metrics, graphs, and CSV files so that the results can be reviewed rather than treated as a black box.

Usable project structure: The folders, outputs, and dashboard controls should be understandable for users without requiring direct code interaction.

3.3 High-Level Pipeline Design

The pipeline starts with a video placed into the raw video folder (either manually or using the UI)

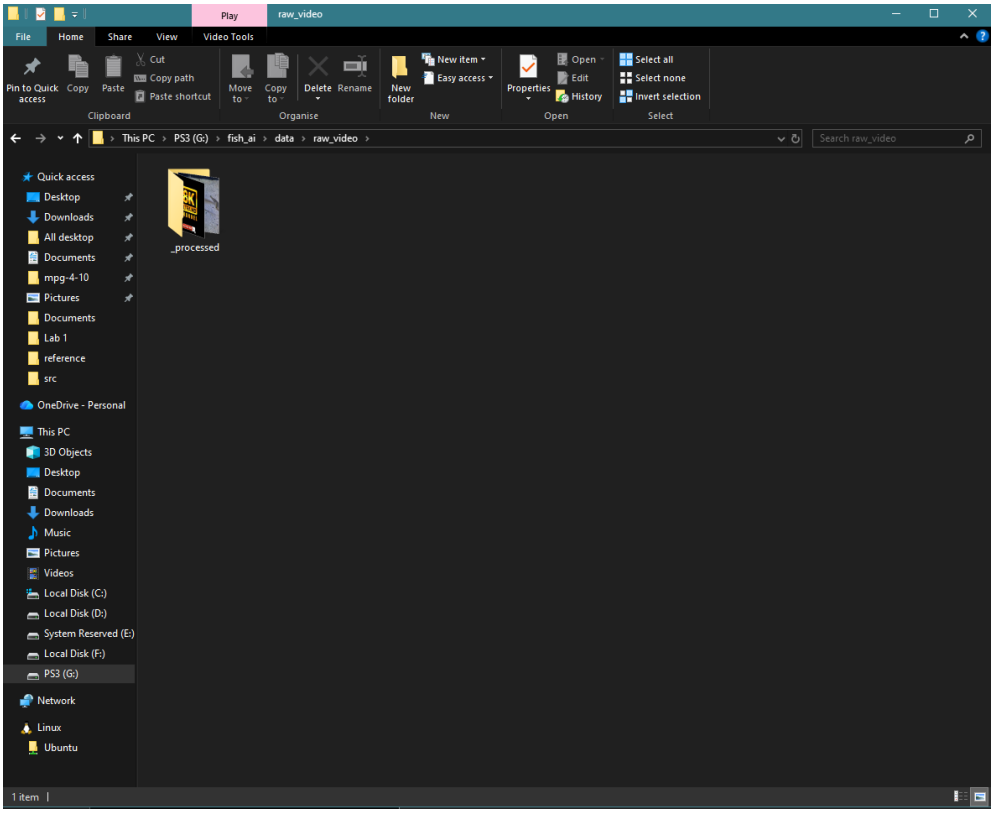


Figure 4: Example of the runs folder that users can place their video files into opened using the UI

The user can then run the video pipeline through the dashboard. This activates run_one_video.py, which coordinates the main processing stages. The system first extracts frames from the video, then selects clearer and less repetitive frames for analysis. These selected frames are passed into crop_fish_from_frames.py, where the trained detector identifies fish and saves each accepted detection as an individual crop. The crops are then processed by filter_crops_quality.py, which removes unsuitable outputs such as blurry, very small, duplicated, or unclear crops. Finally, cluster_fish.py groups the remaining crops into visually similar folders for manual review. After the pipeline has finished, the user can inspect the results through the “Open Video Runs” or “Open Latest Video Run Folder” buttons.

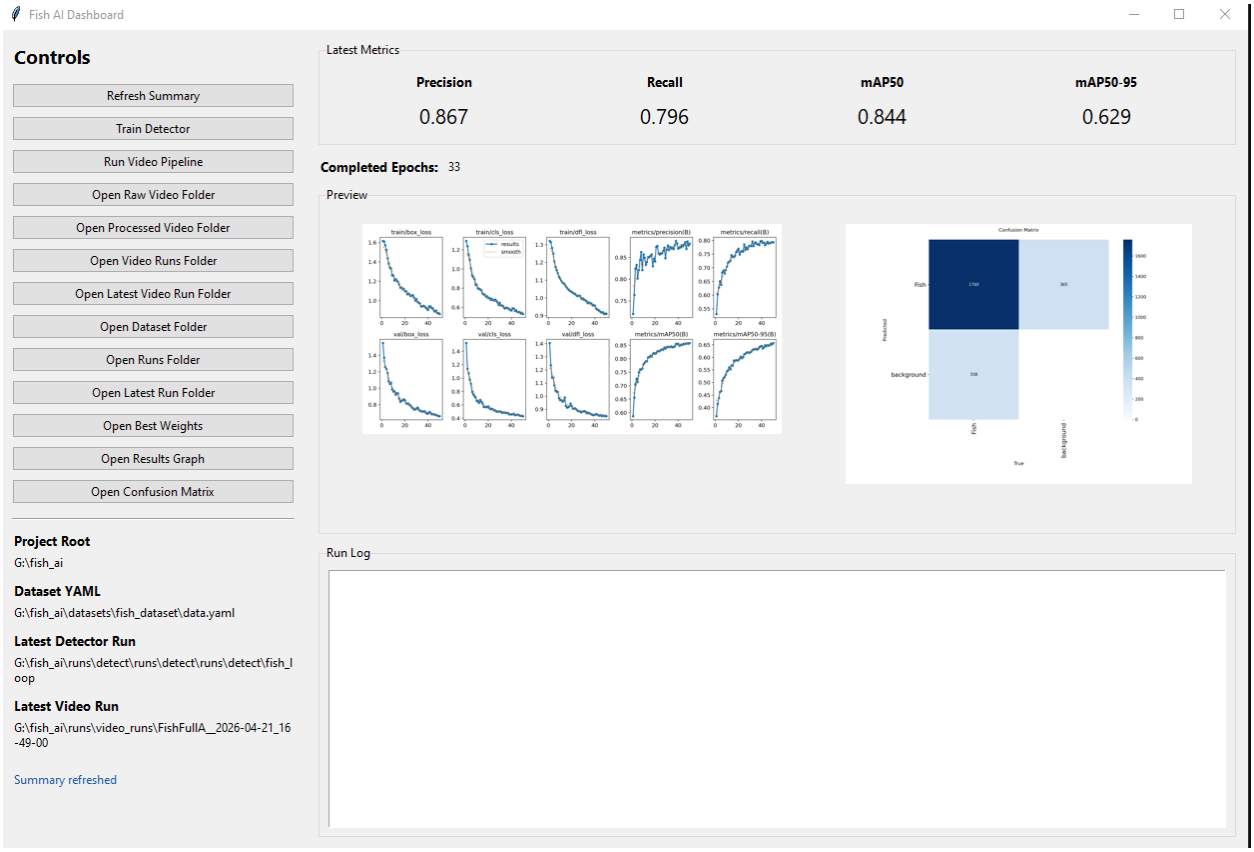


Figure 5: The UI of the app that allows the users to control the model

Additionally there are options on the UI that allow users to interact with the model on a deeper level, such as “Train Detector” that allows users to continue to refine the AI. Users are even able to change the training data using “Open Dataset Folder” to tailor the training to their fish.

3.4 System Architecture

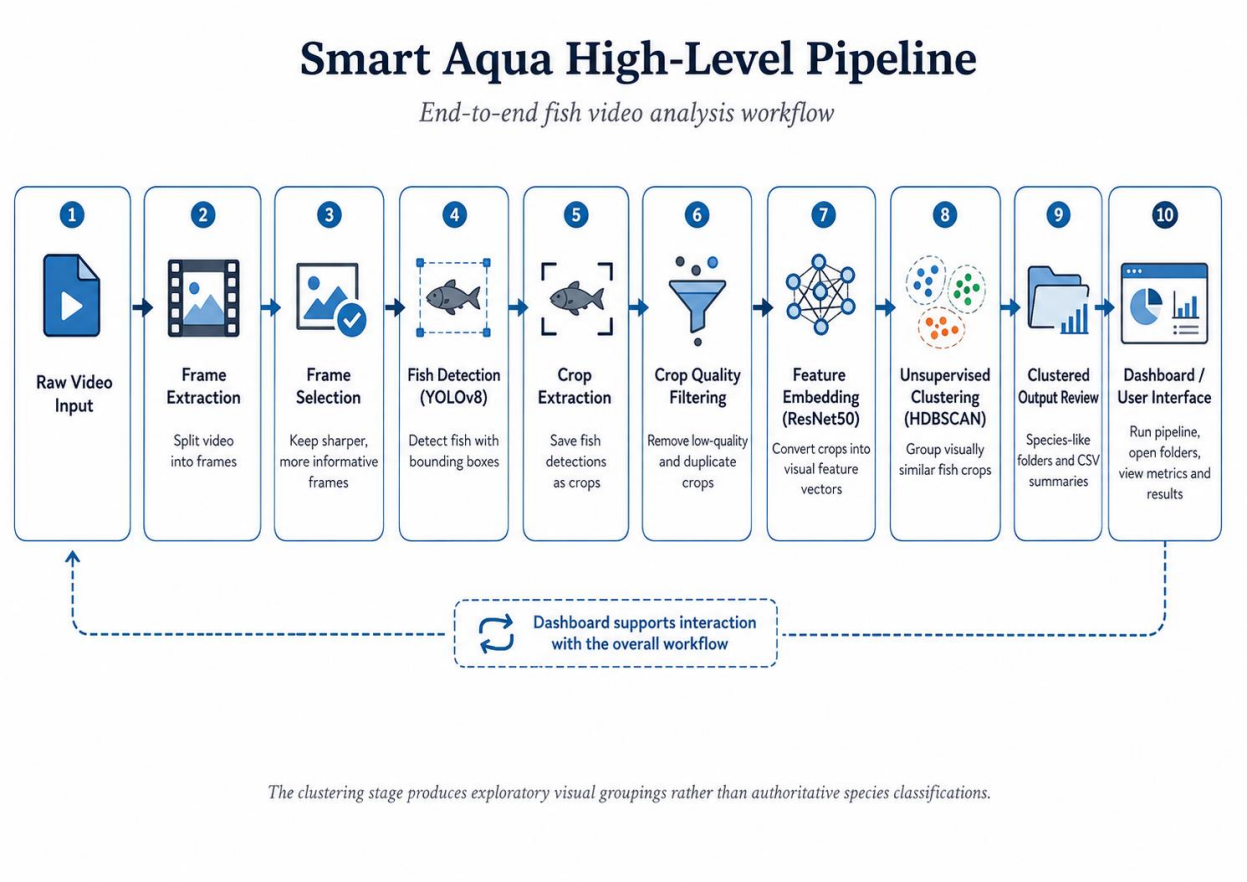


Figure 6: High-level Smart Aqua system architecture showing the pipeline from raw video input to clustered output review and dashboard interaction.

Figure **6** shows the overall architecture of Smart Aqua. The system is structured as a modular pipeline where each stage produces an output that becomes the input for the next stage. This makes the workflow easier to inspect, test, and improve because the outputs from frame extraction, detection, crop filtering, and clustering can all be reviewed separately. The dashboard acts as the user-facing layer of the system, allowing the pipeline to be run and the main outputs to be accessed without requiring direct interaction with the Python scripts.

3.5 Design Rationale

The design of Smart Aqua was based around keeping the system lightweight, repeatable, and easy to use. A single-class detector was chosen because the main aim of the project was to detect fish reliably rather than classify every fish species. This reduced the complexity of the dataset and made the detector easier to train, as every labelled fish could be placed into one shared “Fish” class. A full species classifier would have required verified species labels, which were not available consistently enough for this project.

Frame selection was included before crop extraction so that the system did not process every frame from the video equally. Video footage can contain repeated, blurry, or unclear frames, so selecting

sharper and more informative frames helped reduce unnecessary processing and improved the quality of the data passed into detection. This also helped keep the pipeline more efficient.

Post-crop quality filtering was added because not every detected crop is useful. Some crops may be too small, blurry, low contrast, duplicated, or affected by poor bounding box placement. Filtering these crops before clustering helped reduce noise and made the final grouped outputs easier to review.

Unsupervised clustering was used instead of supervised species classification because the project did not have a reliable labelled species dataset. The clustering stage groups visually similar crops together, but it does not claim to identify species with certainty. This was a suitable design choice because it still adds value after detection while keeping the system within the practical limits of the project.

A simple desktop dashboard was chosen instead of a complex web application because the main purpose was to make the workflow easier to run, review, and demonstrate. The dashboard allows users to access key folders, run pipeline stages, view metrics, and open results without needing to interact directly with the Python scripts. This supports the project aim of creating a practical workflow rather than only a set of disconnected technical experiments.

4. Methodology and Implementation

4.1 Development Approach

Stage	Script / Component	Main output
Detector training	train_detector_only.py	best.pt, metrics, graphs
Frame extraction	extract_frames.py	extracted frames
Frame selection	select_frames_for_labeling.py	selected clearer frames
Crop extraction	crop_fish_from_frames.py	fish crop images + crops.csv
Crop filtering	filter_crops_quality.py	filtered crops + filter_report.csv
Clustering	cluster_fish.py	cluster folders + CSV summaries
Dashboard	UI	buttons, folder access, metrics preview

Iterative refinement was important because the model and pipeline were improved through repeated testing, inspection, and adjustment (Xue et al., 2025). The project began with research into object detection and where to gather fish videos to be analysed, followed by the creation of a basic training pipeline for the fish detector. Once the first detector was working, the system was built around it, including frame extraction, crop extraction, crop filtering, clustering, and finally the dashboard interface.

This approach was useful because the project depended heavily on the quality of the dataset and the practical performance of the pipeline. After each training run or pipeline test, the outputs were inspected to identify problems such as missed fish, false detections, unclear crops, repeated frames, or low-quality clustered outputs.

These issues were then used to guide the next stage of development, such as adjusting dataset selection, improving crop filtering, changing confidence thresholds, or refining how outputs were organised.

The detector was trained and evaluated mainly using precision, recall, mAP50, and mAP50-95, while the pipeline was assessed by checking whether the outputs were useful for later review in the actual results folders. This meant that development was not only focused on improving numerical model performance, but also on whether the full workflow produced clear, usable results. As a result, Smart Aqua developed from a basic detector into a more complete end-to-end system that could process video, detect fish, extract crops, organise outputs, and support user interaction through the dashboard.

```
EXTRACT_EVERY_N_FRAMES = 2
SELECT_MAX_FRAMES = 1200
SELECT_MIN_SHARPNESS = 15.0
SELECT_MIN_SCENE_DELTA = 0.14
SELECT_BACKFILL = False
SELECT_MIN_FRAME_GAP = 12

CROP_CONF = 0.30
CROP_IMGSZ = 640
CROP_PAD = 0.10
CROP_MIN_SIZE = 40
CROP_MAX_IMAGES = 4000

FILTER_MIN_LONG_SIDE = 96
FILTER_MIN_AREA = 96 * 96
FILTER_MIN_SHARPNESS = 20.0
FILTER_MIN_CONTRAST = 12.0
FILTER_MIN_ASPECT_RATIO = 0.30
FILTER_MAX_ASPECT_RATIO = 3.50
FILTER_DEDUPE_HAMMING = 5

CLUSTER_USE_COLOR_HIST = True
CLUSTER_USE_SIZE_FEATS = True
CLUSTER_WHITE_BALANCE = True
CLUSTER_USE_UMAP = False
CLUSTER_UMAP_DIM = 32
CLUSTER_UMAP_NEIGHBORS = 10
CLUSTER_UMAP_MIN_DIST = 0.05
CLUSTER_MIN_CLUSTER_SIZE = 8
CLUSTER_MIN_SAMPLES = 5

ARCHIVE_PROCESSED_VIDEO = True
```

Figure 7: An example of the pipeline configuration values that were adjusted during development, including frame selection, crop filtering, detector confidence, image size, and clustering settings.

4.1.1 Project Management and Iteration

The project was managed using an iterative agile approach, supported by a Gantt chart, sprint planning, supervisor feedback, and ongoing review of technical outputs. This was suitable for Smart Aqua because the project depended heavily on experimentation. The quality of the dataset, detector performance, crop outputs, and clustering results could not be fully predicted at the beginning, so the system needed to be refined through repeated testing and adjustment.

The original project plan included broader ideas such as fish health detection, invasive species identification, and population tracking. However, as development progressed, it became clear that the available dataset, time, and hardware were better suited to a more focused and reliable workflow. The project scope was therefore refined towards a single-class fish detector, crop extraction, quality filtering, unsupervised clustering, and dashboard-based review. This helped keep the project achievable while still producing a complete technical artefact.

Each development stage produced an output that could be reviewed before moving to the next stage. For example, dataset versions were compared, detector metrics were inspected, crops were checked for quality, cluster folders were reviewed visually, and dashboard controls were tested during integration. Issues found during one stage, such as missed fish, unclear crops, false detections, or repeated frames, were then used to guide later improvements.

Version control and file organisation were also important during development. Source code, configuration files, training outputs, screenshots, and report evidence were kept organised so that the project could be reviewed and reproduced. Generated files such as large frame folders, crop outputs,

and training runs were treated separately from the core source code to keep the repository manageable.

The full agile sprint plan, Gantt chart, and risk assessment are included in the appendices. These show how the project was planned, adapted, and controlled throughout development.

4.2 Dataset Creation and Preparation

The dataset creation stage was one of the most important parts of the project because the detector could only learn from the quality and consistency of the images and annotations it was given. The aim was to create a custom fish dataset that contained clear examples of fish in different positions, sizes, lighting conditions, densities, and backgrounds. This was important because the final detector needed to work on video footage where fish may overlap, appear at different distances, or be partially hidden by the aquarium environment.

The dataset preparation process involved collecting video footage, extracting usable frames, manually annotating fish, applying preprocessing and augmentation, and exporting the final dataset in a YOLO-compatible format. Roboflow reported 10,945 annotations across the single “Fish” class, with an average of approximately 11 annotations per image before the augmented export was generated. This meant the detector was trained to identify fish as a general object class rather than distinguish between individual species.

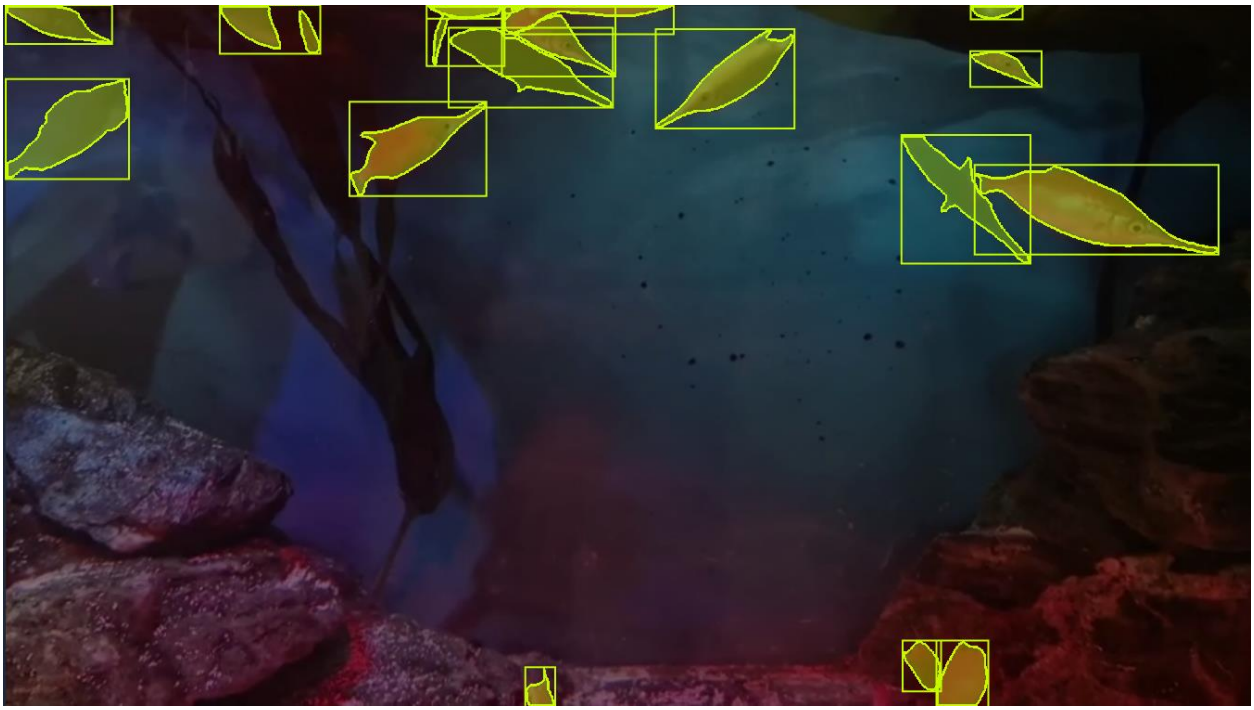


Figure 8: Example annotated training frame showing multiple fish labelled with bounding boxes for the single “Fish” class.

4.2.1 Data Sources

The original plan was to collect fish footage directly from the Plymouth Aquarium, as this would have provided relevant video data from a realistic aquarium environment. However, this footage was not available during development, so alternative data sources were used. These included fish videos from online platforms and additional footage recorded on a mobile phone. This allowed the project to continue while still producing a custom dataset suitable for training a fish detector.

The collected videos were used as the starting point for frame extraction. Instead of using every frame from the videos, extracted frames were reviewed and selected based on suitability. Important selection factors included fish visibility, frame clarity, lighting, fish density, and variation in appearance. This helped reduce repeated or unclear examples and ensured that the dataset contained a wider range of useful training images.

4.2.2 Annotation Strategy

The selected frames were uploaded to Roboflow for manual annotation. Each visible fish instance was labelled using a bounding box and assigned to the single class “Fish”. This single-class strategy was chosen because the purpose of the detector was to locate fish reliably rather than classify individual species. Using one shared class also reduced the risk of incorrect species labels and kept the dataset practical within the project timeframe.

A valid fish instance was treated as a fish that was visible enough to be identified and enclosed by a bounding box. Fish that were extremely unclear, heavily obscured, or not reliably identifiable were avoided where possible, as including them could reduce annotation quality. Consistency was important because the model needed to learn the relationship between fish features and bounding box locations. If fish were missed, labelled inconsistently, or boxed inaccurately, the detector could learn unreliable patterns.

The final dataset contained 10,945 annotations across one class, with an average of approximately 11 annotations per image. This reflected the nature of the footage, where multiple fish could appear in the same frame. This made annotation quality especially important because crowded scenes increased the chance of overlapping fish, partial occlusion, and missed labels.

4.2.3 Dataset Cleaning

Dataset cleaning was carried out to reduce the amount of confusing or low-quality data used during training. Frames were reviewed before annotation so that blurry, repeated, or unsuitable images could be removed. This was important because repeated frames could reduce dataset variety, while unclear frames could introduce noise into the training process.

Roboflow was also used to prepare the dataset before export. Auto-orientation was applied, and images were resized to fit within 512 × 512 pixels. This helped standardise the input format for training and matched the image size used by the detector. Augmentation was then applied to the training images to increase variation in the dataset. The augmentations included cropping, brightness changes, exposure changes, blur, noise, and motion blur. These transformations were useful because underwater and aquarium footage can vary in lighting, sharpness, and movement.

The dataset summary showed 3,240 training images and 272 validation images after augmentation. This difference occurred because augmentation was applied to the training set, while the validation set was kept unaugmented. This was useful because the training data became more varied, while the validation data remained closer to unseen real examples for evaluation.

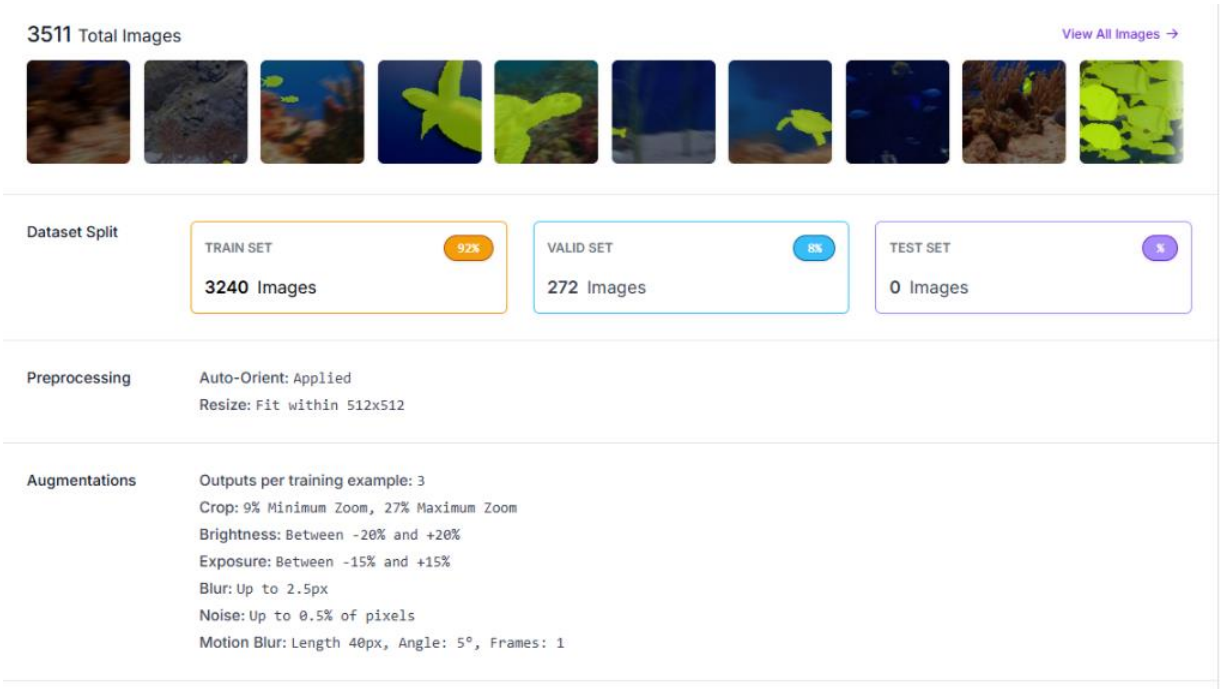


Figure 9: Roboflow dataset summary showing preprocessing, augmentation settings, and the exported training and validation image counts.

4.2.4 Train/Validation Split

Before augmentation, the final dataset was split into 80% training data and 20% validation data, with 1,080 images used for training and 272 images used for validation. The training set was used to teach the model how to detect fish, while the validation set was used to measure how well the model performed on images it had not directly trained on.

After augmentation, the exported training set increased to 3,240 images because Roboflow generated three outputs per training example. The validation set remained at 272 images because it was not augmented. This was intentional, as validation data should remain closer to real unseen data so that model performance can be evaluated more fairly.

This split was important because the detector needed to be assessed on more than its ability to memorise the training examples. The validation set allowed performance to be measured using precision, recall, mAP50, and mAP50-95, which helped show whether the detector was generalising to unseen frames.

The model has been put through multiple iterations with varying sizes and databases due to the nature of the refinement of the model. While this data was constantly being changed and updated there are 4 major changes to the model that have been recorded.

Dataset version	Total images	Train	Valid
V1	300	240	60
V2	490	392	98
V3	1138	910	228
V4	3512	3240	272

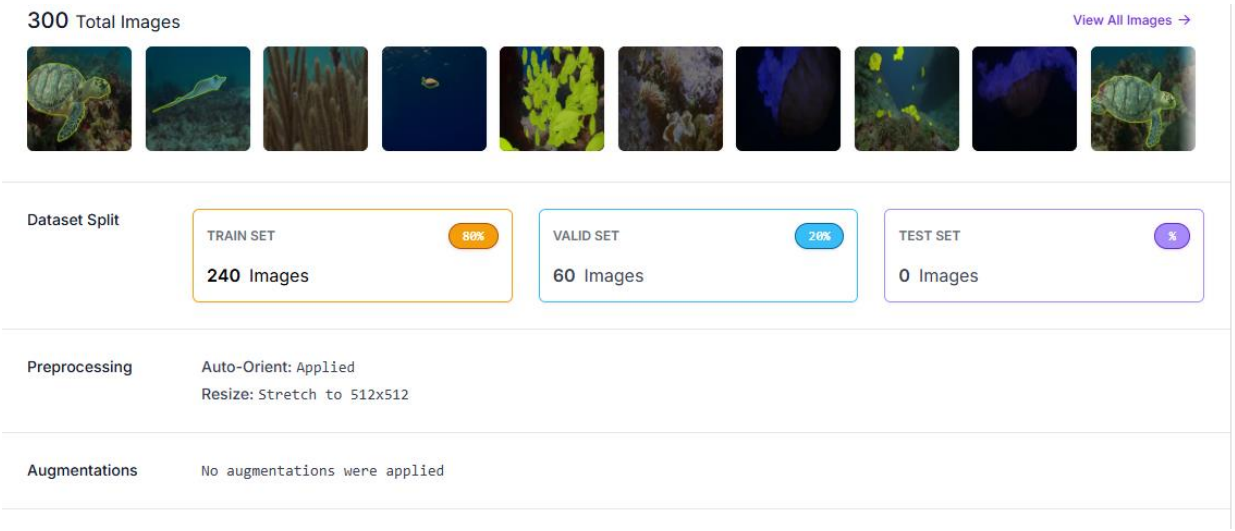


Figure 10: V1 dataset used for comparison with the final dataset version.

4.3 Detector Training Implementation

The fish detector was trained using the train_detector_only.py script. This script was responsible for training the single-class YOLOv8 detector used by the rest of the Smart Aqua pipeline. The detector was trained using the custom Roboflow dataset exported in YOLO format, with the dataset path set to datasets/fish_dataset/data.yaml. The training run was saved under the fish_loop run name so that outputs such as weights, results graphs, and validation artefacts could be kept in a consistent location.

The detector was initialised from yolov8m.pt when no previous trained model was available. However, if a previous best checkpoint already existed, the script automatically continued training from best.pt instead. This was useful because training could be continued across multiple runs without losing the strongest previous model. It also supported the iterative development approach used in the project, where training results were reviewed before deciding whether to continue refining the detector.

```python
# ---------------- CONFIG ----------------
FISH_DATA = "datasets/fish_dataset/data.yaml"
PROJECT_DET = "runs/detect/runs/detect"
FISH_RUN = "fish_loop"

DET_MODEL_START = "yolov8m.pt"
DEFAULT_EPOCHS = 50
DEFAULT_IMGSZ = 512
DEFAULT_BATCH = 8
# ----------------------------------------
```

Figure 11: code snippet of the model's configuration as it trained

The main training configuration used 50 epochs, an image size of 512, and a batch size of 8. These settings were chosen to balance training quality with the available hardware. A larger image size can preserve more visual detail, which is useful for fish detection, but it also increases processing cost. The batch size was kept relatively low to make training more manageable on the available machine.

```python
def train_detector_once(epochs: int, imgsz: int, batch: int) -> Path:
    run_dir = Path(PROJECT_DET) / FISH_RUN
    best_path = run_dir / "weights" / "best.pt"

    model_path = str(best_path) if best_path.exists() else DET_MODEL_START
    print(f"\n=== TRAINING FISH DETECTOR ===")
    print(f"Starting from: {model_path}")
    print(f"Dataset:       {FISH_DATA}")
    print(f"Run folder:    {run_dir}")
    print(f"Epochs:        {epochs}")
    print(f"Image size:    {imgsz}")
    print(f"Batch:         {batch}\n")

    model = YOLO(model_path)
    model.train(
        data=FISH_DATA,
        imgsz=imgsz,
        epochs=epochs,
        batch=batch,
        project=PROJECT_DET,
        name=FISH_RUN,
        exist_ok=True,
    )
```

Figure 12: code snippet of checkpoint selection logic

This checkpoint logic meant that the training process used the best previously saved model where possible. In practice, this allowed the detector to build on earlier training runs rather than restarting

from the base YOLOv8 model each time. The final model used by the pipeline was stored as best.pt, which represents the checkpoint with the strongest validation performance during training.

The training script also saved outputs into the YOLO run folder. These outputs included the trained weights, training curves, validation metrics, and confusion matrix images. The best.pt file was used as the main detector for crop extraction, while graphs such as results.png and the confusion matrix were used later to evaluate training behaviour and model performance. This made the detector training process repeatable and provided evidence for the evaluation chapter.

4.4 Frame Extraction and Selection

The first stage of the video pipeline was frame extraction. This was needed because the detector works on images rather than directly analysing a full video file. The extract_frames.py script opens the input video with OpenCV and saves individual frames to an output folder. OpenCV was used because it provides practical tools for video and image processing, including reading frames from video and applying image analysis operations (Bradski and Kaehler, 2010). The extraction interval is configurable through the --every argument, meaning the pipeline can save every frame or skip frames depending on the amount of data needed. In the final pipeline configuration, frames were extracted at a regular interval so that the video could be processed without creating an unnecessarily large number of repeated images.

```
def main():
    parser = argparse.ArgumentParser(description="Extract frames from a single video.")
    parser.add_argument("--video", required=True, help="Path to input video file")
    parser.add_argument("--out", required=True, help="Output folder for extracted frames")
    parser.add_argument("--every", type=int, default=1, help="Save every N frames (default: 1)")
    args = parser.parse_args()
```

Figure 13: code snippet of configurable frame extraction interval

After extraction, a frame selection stage was used to reduce the number of low-quality or repeated frames passed into the rest of the pipeline. This was important because aquarium video footage can contain many near-identical frames, blurred movement, and frames where fish are not clearly visible. Passing all extracted frames into the detector would increase processing time and could produce unnecessary or low-quality crop outputs.

Frame quality was assessed partly through a sharpness score. The select_frames_for_labeling.py script calculates sharpness by converting each frame to grayscale and measuring the variance of the Laplacian. Frames below the minimum sharpness threshold are rejected before selection. This helped reduce the number of blurred frames used later in the pipeline.

```
def sharpness_score(bgr: np.ndarray) -> float:
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    return float(cv2.Laplacian(gray, cv2.CV_64F).var())
```

Figure 14: code snippet of sharpness score used during frame selection

The selection stage also considered visual diversity. Each frame was converted into a small grayscale signature so that the script could compare how similar it was to frames already selected. A minimum scene difference was used to avoid selecting frames that looked too similar, while a minimum frame gap prevented the system from choosing frames that were too close together in the video. This made the selected frame set more varied and useful for detection.

```
too_close_in_time = any(
    s.frame_idx >= 0 and info.frame_idx >= 0 and abs(info.frame_idx - s.frame_idx) < min_frame_gap
    for s in selected
)
if too_close_in_time:
    continue

min_dist = min(signature_distance(info.signature, s.signature) for s in selected)
if min_dist >= min_scene_delta:
    selected.append(info)
```

Figure 15: code snippet of frame gap and visual diversity selection

This stage improved the efficiency of the pipeline by reducing repeated inputs before detection. It also improved the quality of later crop extraction because clearer and more visually distinct frames were more likely to produce useful fish detections. Overall, frame extraction and selection acted as a preparation stage that converted raw video into a smaller and more useful set of images for the detector.

4.5 Crop Extraction

The crop extraction stage was used to convert detected fish from full video frames into individual fish crop images. This was important because the later filtering and clustering stages work on cropped fish images rather than full frames. The crop_fish_from_frames.py script loads the trained YOLO detector weights, runs prediction on the selected frames, and saves each accepted detection as a separate image crop.

The script includes several configurable settings, including the confidence threshold, inference image size, maximum number of images to process, bounding box padding, and minimum crop size. These settings were useful because they allowed the crop extraction stage to be adjusted depending on the quality of the video and the detector output. For example, a lower confidence threshold can increase the number of fish crops found, but may also increase false positives, while a higher threshold may produce cleaner crops but miss some fish.

```python
def main() -> None:
    parser = argparse.ArgumentParser(description="Detect fish in frames and save crops.")
    parser.add_argument("--source", required=True, help="Folder containing frames")
    parser.add_argument("--out", required=True, help="Output folder for fish crops")
    parser.add_argument("--model", required=True, help="Path to fish detector weights (.pt)")
    parser.add_argument("--conf", type=float, default=0.25, help="Detection confidence threshold")
    parser.add_argument("--imgsz", type=int, default=640, help="Detector inference image size")
    parser.add_argument("--max_images", type=int, default=2000, help="Max number of frames to process")
    parser.add_argument("--pad", type=float, default=0.15, help="BBox padding fraction")
    parser.add_argument("--min_crop", type=int, default=32, help="Skip crops smaller than this (pixels)")
    parser.add_argument(
        "--csv_path",
        default="",
        help="Optional CSV path for crop metadata. Defaults to <out>/crops.csv",
    )
    args = parser.parse_args()
```

Figure 16: code snippet of crop extraction configuration

The detector is then run on each selected frame using the chosen image size and confidence threshold. If fish are detected, the script reads the bounding box coordinates for each detection and uses them to cut the fish region from the original frame. This connects the trained detector directly to the downstream crop-based workflow.

```python
results = model.predict(source=img, imgsz=args.imgsz, conf=args.conf, verbose=False)
r = results[0]

if r.boxes is None or len(r.boxes) == 0:
    continue
```

Figure 17: code snippet of running YOLO prediction on selected frame

Padding is added around each bounding box before cropping. This was included so that the saved crop did not cut too tightly around the fish, which could remove important visual features such as fins or tails. The padded coordinates are also clamped to the image boundaries so that the crop does not go outside the frame.

```python
for i, box in enumerate(r.boxes):
    x1, y1, x2, y2 = map(float, box.xyxy[0].tolist())
    det_conf = float(box.conf[0].item()) if getattr(box, "conf", None) is not None else 0.0

    bw = x2 - x1
    bh = y2 - y1
    x1 = int(max(0, x1 - bw * args.pad))
    y1 = int(max(0, y1 - bh * args.pad))
    x2 = int(min(w, x2 + bw * args.pad))
    y2 = int(min(h, y2 + bh * args.pad))
```

Figure 18: code snippet of bounding box passing and boundary checks

The script also checks the crop size before saving. Very small detections are skipped because they are less useful for later visual comparison and may represent unclear objects or poor detections. Accepted crops are then saved as image files, and metadata such as the original frame path, bounding box coordinates, confidence score, crop width, and crop height is written to a CSV file. This made the crop extraction stage easier to inspect and helped keep the outputs traceable.

4.6 Crop Quality Filtering

The crop quality filtering stage was added to reduce noisy or unsuitable crop outputs before clustering. Although the detector was able to find fish in frames, not every saved crop was useful for later visual comparison. Some crops could be too small, blurry, low contrast, duplicated, or shaped in a way that suggested a poor detection. Passing these crops directly into the clustering stage would make the final groups less reliable and harder to review.

The filtering script assessed each crop using size, area, aspect ratio, sharpness, contrast, and near-duplicate detection. Sharpness was measured using the variance of the Laplacian, while contrast was measured using the standard deviation of the grayscale image. Near-duplicates were identified using a difference hash and Hamming distance comparison.

```
def main() -> None:
    parser = argparse.ArgumentParser(description="Filter fish crops by quality and remove near-duplicates.")
    parser.add_argument("--source", required=True)
    parser.add_argument("--out", required=True)
    parser.add_argument("--min_long_side", type=int, default=96)
    parser.add_argument("--min_area", type=int, default=96 * 96)
    parser.add_argument("--min_sharpness", type=float, default=20.0)
    parser.add_argument("--min_contrast", type=float, default=12.0)
    parser.add_argument("--min_aspect_ratio", type=float, default=0.20)
    parser.add_argument("--max_aspect_ratio", type=float, default=5.00)
    parser.add_argument(
        "--dedupe_hamming",
        type=int,
        default=4,
        help="Near-duplicate threshold for dHash. Lower = stricter dedupe.",
    )
```

Figure 19: code snippet of crop quality filter threshold

Each crop is either copied into the filtered output folder or rejected with a reason recorded in filter_report.csv. This made the filtering stage more transparent because rejected crops could be reviewed rather than silently removed. Overall, this stage improved the quality of the data passed into clustering and helped make the final grouped crop folders easier to interpret.

4.7 Clustering Stage

The clustering stage was used to organise the filtered fish crops into visually similar groups. This stage was not designed to provide confirmed species classification. Instead, it was used as a grouping method so that users could review similar-looking fish crops more easily after detection and filtering.

The cluster_fish.py script uses a pretrained ResNet50 model as a feature extractor. The final classification layer is removed, so the model does not assign a class label to the crop. Instead, each fish crop is converted into a numerical feature embedding that represents its visual appearance. These embeddings are then normalised before clustering. This allowed the system to compare fish crops based on visual similarity without needing a fully labelled species dataset.

The clustering script also includes optional features that can be added to the ResNet50 embedding. These include HSV colour histograms, crop size features, white balancing, and optional UMAP dimensionality reduction. These options were useful because fish similarity may depend not only on shape, but also on colour, size, and lighting. However, these features were treated as supporting features rather than confirmed species information.

The final grouping was performed using HDBSCAN. HDBSCAN was suitable because it can identify dense groups without needing the number of clusters to be set in advance, while also allowing unclear or isolated points to be treated as outliers (Stewart and Al-Khassaweneh, 2022). This was important because the number of visually different fish groups in a video may change depending on the footage. Crops that did not fit clearly into a cluster were placed into an unknown cluster group, while accepted clusters were saved into folders named species_001, species_002, and so on. These folder names should be understood as species-like visual groups rather than confirmed biological species.

```
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=args.min_cluster_size,
    min_samples=args.min_samples,
    metric="euclidean",
)
labels = clusterer.fit_predict(X)
```

Figure 20: code snippet of HDBSCAN clustering applied to the extracted crop features

The script also creates clusters_summary.csv and cluster_map.csv. These files make the clustering output easier to inspect because they record the number of crops in each cluster and map each source crop to its copied output location. This made the clustering stage more transparent and helped users review the results after the pipeline had finished.

4.8 Dashboard UI

The dashboard was developed to make the Smart Aqua workflow easier to run, review, and demonstrate. Without the dashboard, the user would need to manually run several Python scripts and locate the correct folders after each stage. The dashboard brings these actions into one interface, making the system more accessible and reducing the chance of user error.

The main controls allow the user to train the detector, run the video pipeline, open raw and processed video folders, access dataset folders, and view the latest run outputs. This was useful during development because results could be checked quickly after each test run. It also made the final system easier to demonstrate, as the complete workflow could be shown through the interface rather than through command-line commands.

The dashboard also displays important model outputs, including the latest metrics, run path, results graph, and confusion matrix. These outputs helped connect the interface to the evaluation process because the user could inspect training performance and pipeline results from one place. This supports repeatable testing because the same buttons and folder links can be used each time the pipeline is run.

Overall, the dashboard acts as the user-facing layer of Smart Aqua. It does not replace the underlying detector or pipeline scripts, but it makes them easier to control and inspect. This supports the project aim of creating a practical end-to-end workflow rather than a collection of separate technical scripts.

5. Experimental Setup and Evaluation

5.1 Evaluation Strategy

The evaluation strategy was designed to assess Smart Aqua as a full pipeline rather than only as a trained detector. This was important because the project aim was not just to detect fish, but to create a practical workflow that could take video footage, extract frames, detect fish, create crops, filter poor-quality outputs, group similar crops, and present results through the dashboard.

The detector was evaluated using precision, recall, mAP50, and mAP50-95. These metrics were chosen because they measure different parts of object detection performance. Precision showed how many predicted fish detections were correct, while recall showed how many true fish were successfully detected. mAP50 gave an overall view of detection performance at a standard IoU threshold, while mAP50-95 provided a stricter measure of bounding box accuracy across multiple IoU thresholds.

The wider pipeline was evaluated through output inspection. This included checking whether the extracted crops contained usable fish images, whether the crop filtering stage reduced unsuitable outputs, and whether the clustering stage grouped visually similar fish crops in a useful way. This was necessary because strong model metrics alone would not prove that the final workflow was practical. A detector could achieve reasonable scores but still produce crops or clusters that are difficult for users to interpret.

The dashboard was also considered as part of the evaluation because it supported repeatable testing and demonstration. The ability to access metrics, folders, result graphs, confusion matrices, and latest video runs from one interface helped show whether Smart Aqua worked as a usable end-to-end system rather than as separate scripts.

5.2 Hardware and Software Environment

Smart Aqua was developed and tested on a local Windows desktop environment. Although the machine included a dedicated GPU, the detector training was carried out on the CPU. As a result, the training and pipeline settings were chosen to remain practical on the available hardware. This influenced choices such as using a batch size of 8, an image size of 512 during training, and keeping the dashboard as a lightweight desktop interface rather than a web-based system.

Component	**Specification**
Operating system	Windows desktop environment
CPU	AMD Ryzen 7 3700X 8-Core Processor
GPU	AMD Radeon RX 7900 XT, available but not used for detector training
Training device	CPU

Component	Specification
Main language	Python
Detection framework	Ultralytics YOLOv8
Image/video processing	OpenCV
Deep learning libraries	PyTorch, Torchvision
Dataset tool	Roboflow
Clustering tools	ResNet50 feature extraction, HDBSCAN, optional UMAP
Dashboard	Tkinter-based desktop UI

PyTorch supported the deep learning components of the workflow, including model execution and feature extraction (Ketkar and Moolayil, 2021). The project was run locally, with datasets, training runs, generated crops, filtered outputs, and clustered folders stored within the project directory. This local setup was suitable for development and testing, although training on CPU increased experiment time and limited how aggressively image size, batch size, and dataset scale could be increased.

5.3 Training Configurations Compared

Several dataset versions were created during development as the detector and pipeline were refined. The earlier versions were useful for testing the training process and checking whether the model could learn the single “Fish” class, but they were limited by dataset size and visual variety. The final V4 dataset was used as the main evaluated version because it contained the strongest mixture of annotated fish examples, augmentation, and validation data.

The same general training approach was used across the main detector runs, with YOLOv8m used as the starting model, an image size of 512, and a batch size of 8. Training was carried out in 50-epoch runs and continued from the best saved checkpoint where available. This allowed the detector to build on previous training rather than restarting from the base model each time.

Run ID	Dataset version	imgsz	batch	Best metrics	Notes
V1	300 images: 240 train / 60 valid	512	8	Not retained	Early dataset used to test the basic training workflow. Limited data variety.

Run ID	Dataset version	imgsz	batch	Best metrics	Notes
V2	490 images: 392 train / 98 valid	512	8	Not retained	Expanded dataset with more examples, but still limited coverage.
V3	1,138 images: 910 train / 228 valid	512	8	Not retained	Larger dataset, but still lacked enough visual variety.
V4	Final Roboflow export: 3,240 train / 272 valid after augmentation	512	8	Precision 0.882, Recall 0.793, mAP50 0.859, mAP50-95 0.657	Main evaluated detector. Used final dataset and continued checkpoint training.

The V4 run was selected as the main final model because it produced the strongest recorded validation performance and was trained using the most complete dataset. The earlier versions still informed the project because they showed that increasing dataset size alone was not enough; the variety and quality of the examples also affected the usefulness of the detector. For this reason, the final dataset focused on clearer fish visibility, more varied backgrounds, and augmentation rather than only increasing the number of images.

5.4 Detector Results

The final detector was evaluated using the validation set from the final dataset version. The strongest recorded results were a precision of **0.882**, recall of **0.793**, mAP50 of **0.859**, and mAP50-95 of **0.657**. These results show that the model was able to detect fish with a good level of accuracy, while still leaving some room for improvement in recall and stricter bounding box precision.

Metric	Final value	Interpretation
Precision	0.882	Most accepted detections were genuine fish.
Recall	0.793	The detector found most fish, but still missed some.
mAP50	0.859	Strong detection performance at the 0.50 IoU threshold.
mAP50-95	0.657	Moderate performance under stricter bounding box accuracy thresholds.

The high precision score suggests that the detector was generally effective at avoiding false positives. This was important for Smart Aqua because false detections would produce irrelevant crops and reduce the quality of the later filtering and clustering stages. The recall score was lower than precision, which suggests that the model was more likely to miss some fish than to incorrectly classify background objects as fish. This is understandable given the difficulty of the footage, where fish could be small, overlapping, partially hidden, or visually similar to parts of the background.

The mAP50 score of 0.859 shows that the detector performed well when predictions were judged at a standard IoU threshold. This means the model was usually able to identify fish and place bounding boxes close enough to the labelled fish locations. The mAP50-95 score of 0.657 is lower because it uses stricter IoU thresholds, making it a more demanding measure of bounding box quality. This was still useful for the project because tighter bounding boxes are important for crop extraction. If boxes are poorly placed, the resulting crops may cut off part of the fish or include too much background.

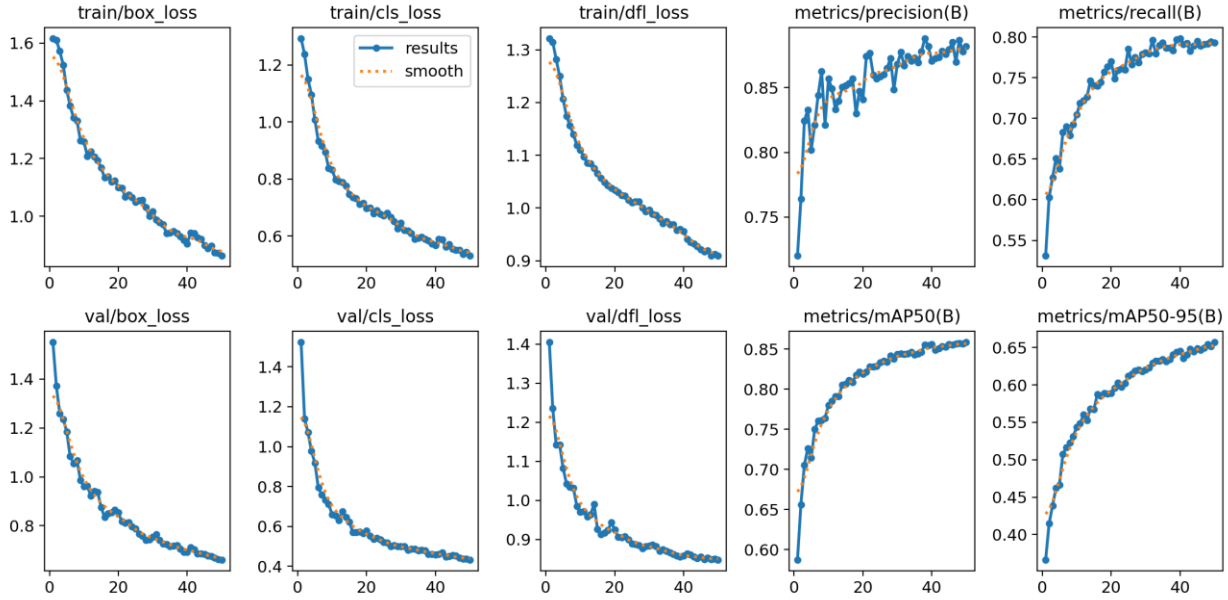


Figure 21: YOLOv8 training results graph showing changes in loss and validation metrics across training.

The training results graph shows how the detector improved during training. The loss curves were used to check whether the model was learning from the dataset, while the precision, recall, and mAP curves were used to judge whether validation performance was improving. These graphs were useful because they showed the training process visually rather than only relying on the final metric values.

There is no single universal threshold that defines a “good” object detector, as acceptable performance depends on the dataset, object type, image quality, and intended use. However, the use of mAP50 and mAP50-95 allows the result to be interpreted against common object detection practice, where mAP50 provides a more tolerant detection measure and mAP50-95 gives a stricter assessment across multiple IoU thresholds (Ultralytics, n.d.; Padilla, Netto and da Silva, 2021). In this context, the final mAP50 of 0.859 suggests strong general detection performance, while the lower mAP50-95 of 0.657 shows that bounding box precision could still be improved.

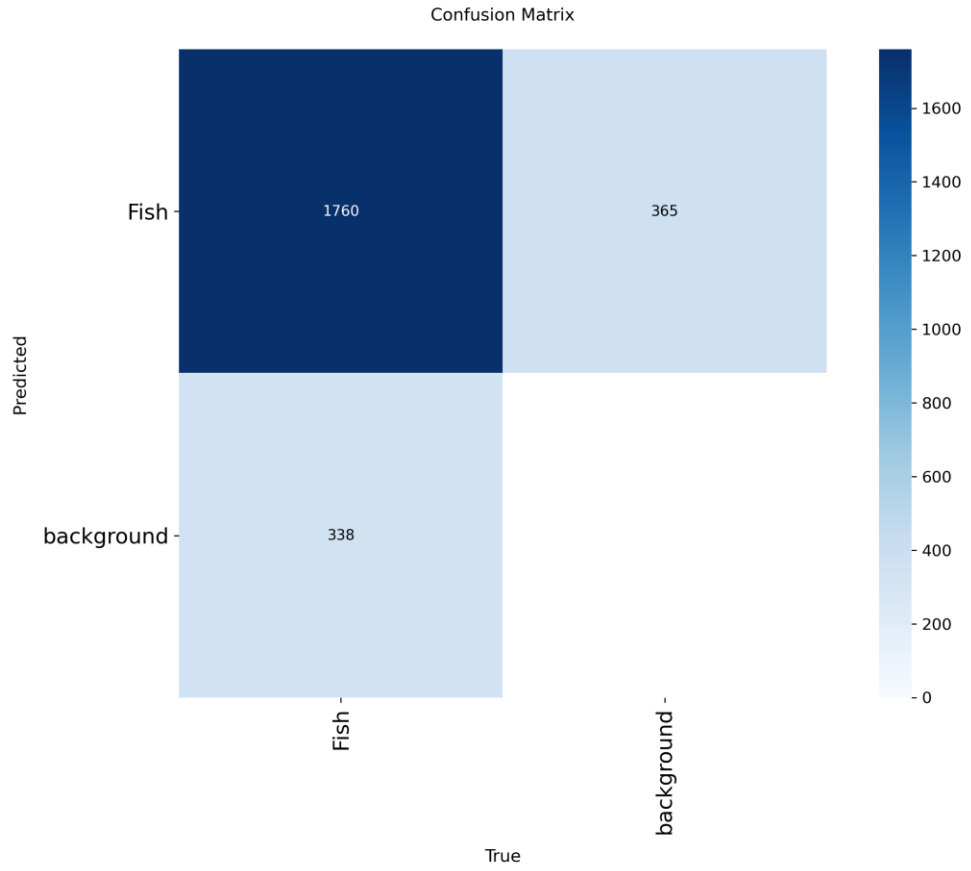


Figure 22: Confusion matrix for the final fish detector.

The confusion matrix was used to inspect how often fish were correctly detected compared with missed or incorrect detections. For a single-class detector, this was useful because the main concern was whether the model could distinguish fish from background objects. The confusion matrix supported the wider evaluation by showing where the detector performed well and where mistakes still occurred.

Overall, the detector results were strong enough to support the rest of the Smart Aqua pipeline. The model produced reliable enough fish detections for crop extraction, filtering, and clustering to be meaningful. However, the lower recall compared with precision shows that future work could focus on finding more missed fish, possibly by lowering the confidence threshold, increasing dataset variety, or further improving annotations.

5.5 Pipeline Output Quality

The pipeline output quality was evaluated by checking whether the outputs from each stage were useful for later review, rather than only relying on the detector metrics. This was important because a model can achieve good validation scores but still produce outputs that are difficult to use in practice. For Smart Aqua, the key question was whether the pipeline produced usable fish crops, reduced poor-quality results, and organised the final outputs in a way that supported manual inspection.

The crop extraction stage was generally effective because detected fish were saved as individual crop images, making it easier to review fish separately from the full video frame. This was useful because full

aquarium frames can contain cluttered backgrounds, overlapping fish, and irrelevant visual information. However, not all crops were equally useful. Some contained partial fish, small detections, background objects, or low-quality images caused by blur or lighting issues.

The crop filtering stage improved the usefulness of the outputs by removing crops that were too small, blurry, low contrast, badly shaped, or near-duplicates. This reduced the amount of noisy data passed into clustering and made the final crop folders easier to inspect. The use of filter_report.csv also made this stage more transparent because rejected crops were recorded with reasons rather than being removed without explanation.

The clustering stage was useful as an exploratory grouping tool. The clusters helped organise visually similar fish crops into folders, which made reviewing the outputs easier than looking through every crop individually. However, the clusters should not be treated as confirmed species labels. Some clusters were visually coherent, while others were affected by similar colours, poor crop quality, or unclear fish features. This was expected because the clustering stage used visual similarity rather than verified species annotations.

The dashboard improved the practical quality of the workflow by making the outputs easier to access. Instead of manually searching through project folders or running each script from the command line, the user could run the pipeline, open output folders, view metrics, and inspect the latest results from one interface. This made the system easier to test and demonstrate, supporting the aim of creating a lightweight end-to-end workflow rather than only a detector model.

Overall, the pipeline produced useful outputs for review, especially when the detector and crop filtering stages worked together. The main limitation was that output quality still depended heavily on the input video and detection quality. Clearer footage produced better crops and more meaningful clusters, while crowded or low-quality footage increased the chance of missed fish, partial crops, and less reliable grouping.

5.6 Error Analysis

The main errors in the pipeline came from difficult video conditions rather than a single failure in the detector. Aquarium footage often contained cluttered backgrounds, moving fish, reflections, low lighting, and overlapping objects. These conditions made detection harder because some fish blended into the background or were only partially visible.

One common issue was missed detections. This was most likely to occur when fish were small, blurred by movement, partly hidden, or close to other fish. Missed detections reduced the number of usable crops available for filtering and clustering. This links to the final recall score, which was lower than precision, showing that the detector was better at avoiding false positives than detecting every fish in the frame.

False positives were another issue, although they were less common than missed detections. Some background areas, rocks, reefs, or other underwater objects could be detected as fish if they had a similar shape or colour. These false positives were important because they could create irrelevant crops, which then had to be removed by the crop filtering stage or reviewed manually.

Crop quality also affected the later stages of the pipeline. Some crops were too small, blurry, too dark, or only contained part of a fish. These outputs made clustering less reliable because the feature extractor had less useful visual information to compare. The filtering stage reduced this problem, but strict filtering also created a trade-off because some borderline useful crops could be removed.

The clustering stage had its own limitations. Because the clusters were based on visual similarity rather than confirmed species labels, fish with similar colours or shapes could be grouped together even if they were not the same species. Likewise, poor crops or unclear fish could be placed into less meaningful clusters or into unknown_cluster. This means the clustering output was useful for review, but not reliable enough to be treated as automatic species identification.

Overall, the main errors showed that Smart Aqua worked best when the source footage was clear and fish were visible. The pipeline was still useful on more difficult footage, but output quality became more dependent on filtering and manual review.

5.7 Comparison Against Objectives

The first objective was to develop a robust fish detector trained on a custom dataset. This was achieved because the project produced a single-class YOLOv8 detector trained on a custom Roboflow fish dataset. The final detector achieved a precision of 0.882, recall of 0.793, mAP50 of 0.859, and mAP50-95 of 0.657, showing that it could identify fish with a good level of accuracy.

The second objective was to build an end-to-end video analysis pipeline from frame extraction to clustered crop output. This was achieved through the completed workflow, which takes raw video, extracts frames, selects useful frames, detects fish, creates crops, filters low-quality crops, and groups the remaining crops into visually similar clusters. This showed that the project developed beyond a standalone model and became a working analysis pipeline.

The third objective was to evaluate detector performance using precision, recall, mAP50, and mAP50-95. This was achieved through the final validation results and training outputs. These metrics allowed the detector to be assessed from multiple perspectives, including correctness of detections, missed fish, and bounding box quality.

The fourth objective was to assess the effectiveness of downstream crop filtering and clustering. This was partially achieved. The filtering stage clearly improved the quality of outputs by removing unsuitable crops, and the clustering stage provided useful visual groupings for manual review. However, because the clusters were not based on verified species labels, they should be treated as exploratory rather than authoritative classifications.

The fifth objective was to develop a lightweight dashboard interface to support repeatable use and demonstration. This was achieved because the dashboard allowed the user to run the pipeline, open folders, view metrics, and access outputs without directly interacting with the scripts. This made the final system easier to use and demonstrate.

Overall, the project met its main objectives. The strongest outcome was the creation of a complete, practical workflow rather than only a trained detector. The main limitation was that the clustering stage supports visual review but does not provide confirmed species classification.

6. Discussion

6.1 Interpretation of Results

The results show that Smart Aqua was most successful as a complete workflow rather than as only a standalone detector. The final detector results were strong enough to support the later stages of the pipeline, with high precision showing that most accepted detections were genuine fish. This was important because false detections would have created poor crops and reduced the usefulness of clustering.

The lower recall shows that the model still missed some fish, especially in difficult frames where fish were small, blurred, overlapping, or similar to the background. However, this was an expected limitation of the footage and dataset. In practice, the detector still produced enough usable detections for crop extraction, filtering, and clustering to function meaningfully.

The most important design choice was using a single-class fish detector instead of trying to classify individual species. This kept the dataset more manageable and allowed the project to focus on reliable fish localisation. The crop filtering stage also added practical value because it reduced noisy outputs before clustering. Without this stage, blurry, duplicated, or very small crops would have made the final clusters harder to interpret.

The clustering stage added value by organising visually similar fish crops into groups for review, but it should not be treated as species identification. Its main contribution was reducing the amount of manual browsing needed after detection. The dashboard then made the whole workflow easier to operate and demonstrate by bringing the main controls, folders, and results into one interface.

Overall, the results suggest that the strongest part of Smart Aqua is the way each stage supports the next. The detector provides the initial fish locations, crop extraction turns those detections into reviewable images, filtering improves the quality of those images, clustering organises them, and the dashboard makes the process easier to use. This supports the project aim of creating a practical, lightweight fish video analysis workflow.

6.2 Trade-Offs

Several trade-offs were identified during the development of Smart Aqua. One of the main trade-offs was between recall and false positives. Lowering the confidence threshold could help the detector find more fish, but it could also increase the number of background objects incorrectly detected as fish. This would create more irrelevant crops and add extra pressure on the filtering stage.

A second trade-off was between image size and processing time. Larger image sizes can preserve more visual detail, which may help with small or partially visible fish, but they also increase training and inference time. This was especially important because the detector was trained on the CPU, meaning very large settings were less practical.

Crop filtering also involved a balance between removing noisy outputs and keeping borderline useful examples. Stricter filtering helped remove blurry, duplicated, or very small crops, but it could also remove some crops that may still have been useful for manual review.

The dashboard involved a final trade-off between simplicity and functionality. A desktop dashboard was easier to build, test, and demonstrate than a more complex web application, but it also limited wider access and advanced features. This was acceptable for the project because the aim was to create a lightweight workflow rather than a full enterprise platform.

6.3 Limitations

Smart Aqua had several limitations. The first was dataset quality and availability. The original plan to collect footage from Plymouth Aquarium was not possible, so the dataset had to be built from alternative sources. Although this allowed the project to continue, it limited control over lighting, camera angle, species variety, and video quality.

A second limitation was that training was carried out on the CPU. This increased training time and limited how far the model settings could be pushed, such as using larger image sizes, bigger batch sizes, or more repeated experiments.

The clustering stage was also limited because it did not use verified species labels. The clusters were based on visual similarity, so they could help organise crops for review, but they could not be treated as confirmed species classifications.

Finally, the system depended heavily on the quality of the input video. Clear footage produced better detections, crops, and clusters, while crowded, blurry, or low-light footage increased missed detections and poor crops. This means the system is useful as a lightweight workflow, but it still requires human review and further testing on broader video sources.

6.4 Legal, Ethical, Social, and Professional Considerations

Smart Aqua raised several legal, ethical, social, and professional considerations. Legally, the main concern was dataset provenance. Because some footage came from online platforms and mobile recordings rather than a controlled institutional source, the data was used only for educational research and was not presented as a commercial dataset. Care was also taken to avoid using personal or sensitive data, as the project focused on fish and aquarium footage rather than identifiable people.

Ethically, the system needed to be presented responsibly. The detector and clustering stage should not be treated as replacing expert judgement. This was especially important for the clustering stage, because the groups are based on visual similarity rather than verified species labels. For this reason, the report presents the clusters as exploratory visual groupings, not confirmed species classifications.

Socially, the project has potential value because it could support fish monitoring by reducing the amount of manual video review needed. However, the system should be seen as a support tool rather than a final decision-making system. Human review remains important, especially when outputs may be used to check fish health, behaviour, or abnormalities.

Professionally, the project aimed to remain reproducible and transparent. Outputs such as crops, filter reports, cluster summaries, metrics, graphs, and confusion matrices were saved so that results could be reviewed rather than treated as a black box. The dashboard also supported repeatable use by allowing the same workflow to be run and inspected through consistent controls.

6.5 What Makes the Final System Valuable

The main value of Smart Aqua is that it combines several separate technical stages into one usable workflow. A trained detector on its own would only show where fish appear in a frame, but Smart Aqua continues beyond detection by extracting crops, filtering poor-quality outputs, grouping visually similar fish, and making the results accessible through the dashboard.

This makes the system more practical because each stage improves the usefulness of the next. Detection provides the bounding boxes, crop extraction creates reviewable fish images, filtering removes noisy outputs, clustering organises the remaining crops, and the dashboard allows the user to run and inspect the workflow without using the command line.

The final system is therefore valuable because it is not just a model result, but a lightweight tool for processing fish video footage. Although it still requires human review and does not provide confirmed species classification, it reduces manual workload and provides a clear foundation for future improvements.

7. Conclusion and Future Work

7.1 Conclusion

Smart Aqua successfully achieved its main aim of creating a lightweight fish video analysis workflow that combines object detection, crop extraction, quality filtering, unsupervised clustering, and dashboard-based output review. The project was not designed as a full enterprise aquaculture system or an authoritative species classifier. Instead, it focused on building a practical end-to-end pipeline that could support fish monitoring by reducing the amount of manual video review required.

The strongest part of the project was the custom-trained YOLOv8 fish detector. Using the final dataset, the detector achieved a precision of 0.882, recall of 0.793, mAP50 of 0.859, and mAP50-95 of 0.657. These results show that the detector was effective at identifying fish in challenging aquarium footage, especially in terms of precision. The lower recall shows that some fish were still missed, particularly in difficult conditions such as cluttered backgrounds, overlapping fish, blur, or low lighting. However, the results were strong enough to support the later stages of the pipeline.

The project also demonstrated that detection becomes more useful when connected to a wider workflow. Detected fish were converted into crops, poor-quality or duplicate crops were filtered, and the remaining images were grouped using ResNet50 feature extraction and HDBSCAN clustering. Although the clusters cannot be treated as confirmed species labels, they provided a useful way to organise visually similar fish crops for manual review. This supported the project's goal of creating an accessible and practical system rather than only producing model metrics.

The dashboard further improved the final system by allowing users to run the pipeline, access folders, view metrics, and inspect outputs without directly interacting with the Python scripts. This made Smart Aqua easier to test, demonstrate, and repeat. Overall, the project shows that a lightweight AI-assisted workflow can be built for fish video analysis using available tools and local hardware. While future work is needed to improve recall, test broader datasets, and add stronger species verification, Smart Aqua provides a solid foundation for practical fish detection and visual review.

7.2 Key Contributions

- A custom-trained fish detector.
- An automated end-to-end video analysis pipeline.
- A post-processing stage for crop quality improvement.
- An unsupervised clustering stage for organising outputs.
- A practical dashboard for managing runs and outputs.

7.3 Future Work

- Improve label cleaning and dataset validation.
- Experiment with higher-recall detector settings.
- Add stronger species classification or human-in-the-loop review.
- Test on broader and more varied video sources.
- Improve the dashboard and reporting automation.

References

1. **IBM** (n.d.) *What is Object Detection?* Available at: https://www.ibm.com/think/topics/object-detection (Accessed: **18 November 2025**).

2. **Kundu, R.** (2023) *YOLO: Algorithm for Object Detection Explained [+Examples].* V7. Available at: https://www.v7labs.com/blog/yolo-object-detection (Accessed: **24 November 2025**).

3. **Lutz, C.G.** (2023) *The rise of AI in aquaculture.* The Fish Site, 1 March. Available at: https://thefishsite.com/articles/the-rise-of-ai-in-aquaculture-artificial-intelligence (Accessed: **2 December 2025**).

4. **Roboflow** (n.d.) *Roboflow: Computer vision tools for developers and enterprises.* Available at: https://roboflow.com/ (Accessed: **9 December 2025**).

5. **Tableau** (n.d.) *What is the history of artificial intelligence (AI)?* Available at: https://www.tableau.com/data-insights/ai/history (Accessed: **15 December 2025**).

6. **Xue, E., Huang, Z., Ji, Y. and Wang, H.** (2025) *IMPROVE: Iterative Model Pipeline Refinement and Optimization Leveraging LLM Agents.* arXiv. Available at: https://arxiv.org/html/2502.18530v1 (Accessed: **7 January 2026**).

7. **Hermens, F.** (2024) *Automatic object detection for behavioural research using YOLOv8. Behavior Research Methods,* 56, pp. 7307–7330. Available at: https://link.springer.com/article/10.3758/s13428-024-02420-5 (Accessed: **12 January 2026**).

8. **Stewart, G. and Al-Khassaweneh, M.** (2022) *An Implementation of the HDBSCAN Clustering Algorithm. Applied Sciences,* 12(5), 2405. Available at: https://www.mdpi.com/2076-3417/12/5/2405 (Accessed: **20 January 2026**).

9. **Bradski, G. and Kaehler, A.** (2010) *ICRA 2010 ROS Tutorial: OpenCV Tutorial.* Available at: https://mirror.umd.edu/roswiki/attachments/Events%282f%29ICRA2010Tutorial/ICRA_2010_OpenCV_Tutorial.pdf (Accessed: **3 February 2026**).

10. **Ketkar, N. and Moolayil, J.** (2021) *Introduction to PyTorch.* In: *Deep Learning with Python.* Berkeley, CA: Apress, pp. 27–91. Available at: https://link.springer.com/chapter/10.1007/978-1-4842-5364-9_2 (Accessed: **16 January 2026**).

11. **Padilla, R., Netto, S.L. and da Silva, E.A.B.** (2021) *A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit*. Electronics, 10(3), 279. Available at: https://www.mdpi.com/2079-9292/10/3/279 (Accessed: 18 January 2026).

12. **Ultralytics** (n.d.) *Performance Metrics Deep Dive*. Available at: https://docs.ultralytics.com/guides/yolo-performance-metrics/ (Accessed: 18 January 2026).

Appendices

Appendix A. User Manual

A.1 Purpose of the User Manual

This user manual explains how to use the Smart Aqua fish detection and clustering system. The system is designed to allow a user to process fish video footage through a complete workflow, including video input, frame extraction, fish detection, crop extraction, crop quality filtering, unsupervised clustering, and output review through the dashboard interface.

The system can be operated through the Fish AI Dashboard, which provides buttons for running the main pipeline stages and opening important folders and results. This means the user does not need to manually run every Python script from the command line during normal use.

A.2 System Overview

Smart Aqua is built as a lightweight end-to-end workflow. The main stages are:

1. Raw fish video is added to the project.
2. The video is split into image frames.
3. Clearer and more useful frames are selected.
4. The YOLOv8 detector identifies fish in the selected frames.
5. Detected fish are cropped into individual images.
6. Poor-quality or duplicate crops are filtered.
7. Remaining crops are grouped using unsupervised clustering.
8. Outputs can be reviewed through folders, CSV files, graphs, and dashboard previews.

The system does not provide confirmed species classification. The clustering stage organises visually similar fish crops into groups, but these should be treated as exploratory visual groupings rather than authoritative species labels.

A.3 Opening the Dashboard

To use the project through the dashboard:

1. Open the Smart Aqua project folder.
2. Locate the dashboard executable or dashboard script.
3. If using the executable version, keep the .exe file inside its original build folder with the _internal folder.
4. Open the dashboard by double-clicking the executable or running the dashboard script.

Important: the executable should not be moved away from its _internal folder. If the user wants quick access from the desktop, a shortcut should be created instead of moving the executable itself.

A.4 Dashboard Layout

The dashboard is divided into several main areas:

- **Control Panel:** contains buttons for training the detector, running the video pipeline, and opening key folders.
- **Latest Metrics:** displays the latest detector performance values, including precision, recall, mAP50, and mAP50-95.
- **Preview Area:** shows the latest results graph and confusion matrix.
- **Project Information Area:** shows the project root, dataset YAML path, latest detector run, and latest video run.
- **Run Log:** provides status information while the system is being used.

This layout allows the user to operate the system and inspect important outputs without needing to search through the project folder manually.

A.5 Dashboard Buttons and Their Functions

Refresh Summary

The **Refresh Summary** button updates the dashboard with the latest available project information. This includes the most recent metrics, latest detector run path, latest video run path, results graph, and confusion matrix.

This button should be used after training the detector or running the video pipeline so the dashboard displays the newest outputs.

Train Detector

The **Train Detector** button starts or continues training the YOLOv8 fish detector. The detector is trained using the dataset defined in the project's dataset YAML file.

The detector training script is designed to continue from the best saved model if one already exists. This means that if best.pt is available, training can build on the previous strongest checkpoint rather than starting again from the base YOLOv8 model.

After training, the main output is usually saved as:

runs/detect/fish_loop/weights/best.pt

This file represents the best detector checkpoint from validation performance and is used later by the crop extraction stage.

Run Video Pipeline

The **Run Video Pipeline** button processes a video through the full Smart Aqua workflow. This is the main button used when the user wants to analyse fish footage.

The pipeline performs the following stages:

1. Extracts frames from the video.
2. Selects clearer and more visually useful frames.
3. Runs fish detection on the selected frames.
4. Saves detected fish as individual crops.
5. Filters low-quality or duplicate crops.
6. Groups remaining crops into visually similar clusters.
7. Saves outputs into a video run folder.

After the pipeline has finished, the user should open the latest video run folder to inspect the results.

Open Raw Video Folder

The **Open Raw Video Folder** button opens the folder where input videos should be placed before running the pipeline.

The user should copy or move their fish video file into this folder before using **Run Video Pipeline**.

Recommended video types include common formats such as:

.mp4
.mov
.avi

The video should be clear enough for fish to be visible. Very blurry, dark, or heavily obstructed footage may reduce detection quality.

Open Processed Video Folder

The **Open Processed Video Folder** button opens the folder where processed video-related outputs may be stored. This is useful for reviewing files that have already passed through part of the video workflow.

Open Video Runs Folder

The **Open Video Runs Folder** button opens the main folder containing previous video processing runs. Each run is usually stored in a separate timestamped folder.

This is useful when comparing multiple pipeline runs or checking older outputs.

Open Latest Video Run Folder

The **Open Latest Video Run Folder** button opens the most recent video run directly. This is usually the most useful folder after pressing **Run Video Pipeline**.

The latest video run folder may contain outputs such as:

extracted frames
selected frames
fish crops
filtered crops
clustered crop folders
CSV summaries

This folder is important because it contains the main practical output of the pipeline.

Open Dataset Folder

The **Open Dataset Folder** button opens the dataset folder used for detector training.

The dataset normally contains:

train/images
train/labels
valid/images
valid/labels
data.yaml

The data.yaml file tells YOLO where the training and validation images are stored and which class labels are being used.

For Smart Aqua, the detector uses a single class:

Fish

Users should be careful when changing this folder, as incorrect dataset structure or missing labels may prevent training from working correctly.

Open Runs Folder

The **Open Runs Folder** button opens the folder where YOLO training outputs are saved. This includes training runs, weights, graphs, confusion matrices, and results files.

This folder is useful when reviewing older detector versions or checking training evidence.

Open Latest Run Folder

The **Open Latest Run Folder** button opens the most recent detector training run. This folder normally contains key training outputs such as:

weights/best.pt
weights/last.pt
results.png
confusion_matrix.png
results.csv

These outputs are useful for evaluating model performance.

Open Best Weights

The **Open Best Weights** button opens or locates the best.pt file. This is the strongest saved detector checkpoint based on validation performance.

The best.pt file is important because it is used by the pipeline to detect fish during crop extraction.

Open Results Graph

The **Open Results Graph** button opens the YOLO training results graph. This graph shows how losses and metrics changed during training.

The graph can help the user inspect:

- training loss,
- validation loss,
- precision,
- recall,
- mAP50,
- mAP50-95.

This is useful for checking whether the detector improved during training.

Open Confusion Matrix

The **Open Confusion Matrix** button opens the latest confusion matrix image. This helps the user inspect how well the detector distinguished fish from background.

For a single-class detector, the confusion matrix is useful for checking correct fish detections, missed fish, and background mistakes.

A.6 Preparing a Video for Processing

Before running the video pipeline:

1. Open the raw video folder using the dashboard.
2. Copy the fish video into the folder.
3. Make sure the video file is not corrupted and can be opened normally.
4. Return to the dashboard.
5. Press **Run Video Pipeline**.

For best results, the video should contain visible fish, reasonable lighting, and as little extreme blur as possible. Crowded scenes, reflections, low lighting, and overlapping fish may reduce detection and clustering quality.

A.7 Running the Full Pipeline

To run the full Smart Aqua workflow:

1. Open the dashboard.
2. Press **Refresh Summary** to check the current project state.
3. Place a video in the raw video folder.
4. Press **Run Video Pipeline**.
5. Wait for the pipeline to complete.
6. Press **Refresh Summary** again.
7. Open the latest video run folder.
8. Review the generated outputs.

The pipeline output should be reviewed manually because the system is designed as a support tool, not as a fully automatic final decision system.

A.8 Reviewing Detector Metrics

The dashboard displays the latest detector metrics:

- **Precision:** how many predicted fish detections were correct.
- **Recall:** how many true fish were successfully found.
- **mAP50:** detection performance at an IoU threshold of 0.50.
- **mAP50-95:** stricter detection performance across multiple IoU thresholds.

A high precision score means the detector is usually correct when it predicts a fish. A lower recall score means some fish may still be missed. This is important when reviewing results because missed detections reduce the number of crops available for filtering and clustering.

A.9 Reviewing Training Outputs

After detector training, the user should inspect the latest training run folder. Important files include:

best.pt
last.pt
results.png

confusion_matrix.png
results.csv

The most important file is best.pt, as this is the model used for detection in the pipeline.

The results.png file shows training progress over time. The confusion matrix shows how often the detector correctly identified fish compared with mistakes involving background.

A.10 Reviewing Crop Outputs

After the video pipeline has run, the user should inspect the generated crop folders.

The crop extraction stage saves individual fish detections as separate image files. These crops are useful because they allow fish to be reviewed separately from the full video frame.

Some crops may be imperfect. For example, a crop may:

- include only part of a fish,
- contain a very small fish,
- include background objects,
- be blurry or low contrast,
- include overlapping fish.

For this reason, the pipeline includes a crop quality filtering stage.

A.11 Reviewing Filtered Crops

The filtered crop folder contains crops that passed quality checks. The filtering stage is designed to remove unsuitable crops before clustering.

The filter checks include:

- crop size,
- crop area,
- sharpness,
- contrast,
- aspect ratio,
- near-duplicate detection.

The system may also save a file called:

filter_report.csv

This file records which crops were kept or rejected and why. This makes the filtering process more transparent and allows the user to review rejected crops if needed.

A.12 Reviewing Clustered Outputs

The clustering stage groups filtered fish crops into visually similar folders.

Example output folders may include:

species_001
species_002
species_003
unknown_cluster

The species_001, species_002, and similar names do not mean the system has confirmed the biological species. They are only labels for visual groupings.

The unknown_cluster folder contains crops that did not clearly fit into a cluster. This may include unclear crops, unusual fish appearances, poor detections, or outliers.

The clustering stage may also produce:

clusters_summary.csv
cluster_map.csv

clusters_summary.csv records how many crops are in each cluster.
cluster_map.csv records where each source crop was copied during clustering.

These files help the user review the cluster outputs more systematically.

A.13 Interpreting Cluster Results Responsibly

The clustering stage should be interpreted carefully. It is based on visual similarity, not verified species labels.

This means:

- similar-looking fish may be grouped together,
- different species with similar colours may appear in the same cluster,
- unclear crops may be placed in the wrong cluster,

- clusters should not be used as confirmed species identification.

The clusters are most useful for reducing manual browsing and helping users inspect similar crops together.

A.14 Retraining the Detector

The detector can be retrained if the user has added or improved training data.

Before retraining:

1. Open the dataset folder.
2. Check that the dataset contains valid train and valid folders.
3. Check that images and labels are correctly matched.
4. Check that data.yaml points to the correct dataset structure.
5. Open the dashboard.
6. Press **Train Detector**.

The training script will use the previous best.pt checkpoint if it exists. If no previous checkpoint is available, it will start from the base YOLOv8 model.

Retraining may take a long time, especially if the system is training on CPU.

A.15 Recommended Workflow for a New User

A new user should follow this workflow:

1. Open the dashboard.
2. Press **Refresh Summary**.
3. Open the raw video folder.
4. Add the video to be analysed.
5. Press **Run Video Pipeline**.
6. Wait for the pipeline to finish.
7. Press **Refresh Summary** again.
8. Open the latest video run folder.

9. Review extracted crops, filtered crops, and cluster folders.
10. Open the results graph and confusion matrix if detector performance needs to be checked.
11. Treat clusters as visual groupings, not confirmed species labels.

A.16 Troubleshooting

The dashboard does not open

Check that the executable has not been moved away from its _internal folder. If using a shortcut, make sure the shortcut points to the original executable location.

The video pipeline does not run

Check that a video file has been placed in the raw video folder. Also check that the video format is supported and that the file can be opened normally.

No fish crops are produced

Possible causes include:

- the detector weights are missing,
- the input video does not contain visible fish,
- the confidence threshold is too high,
- the video quality is too poor,
- the wrong folder was used as input.

The user should check the latest detector run, confirm that best.pt exists, and test with clearer footage.

Too many false detections are produced

Possible causes include:

- background objects look similar to fish,
- the confidence threshold is too low,
- the detector needs more varied training data,

- the footage contains cluttered aquarium backgrounds.

The user can review false positives and consider improving the dataset or adjusting the detection confidence threshold.

Crops are too blurry or low quality

This may be caused by fast fish movement, low lighting, poor video quality, or weak detections. The crop filtering stage should remove many of these, but the user may need to use clearer footage for better results.

Clusters do not look like species groups

This is expected in some cases. The clustering stage groups crops by visual similarity, not verified species labels. Poor crop quality, similar colours, or unclear fish features can reduce cluster quality.

Training takes too long

Training was carried out on CPU in the project environment, so training can be slow. To reduce training time, the user can use manageable image sizes, batch sizes, and training runs. More powerful hardware would allow larger experiments.

A.17 Good Practice Notes

Users should follow these good practice steps:

- Keep the project folder structure unchanged where possible.
- Do not move generated files manually unless they are being archived.
- Keep best.pt and training outputs backed up.
- Keep the dataset organised with matching image and label files.
- Use the dashboard to open folders rather than manually searching where possible.
- Review outputs manually before drawing conclusions.
- Avoid treating clusters as confirmed species classifications.
- Record important runs and results for reproducibility.

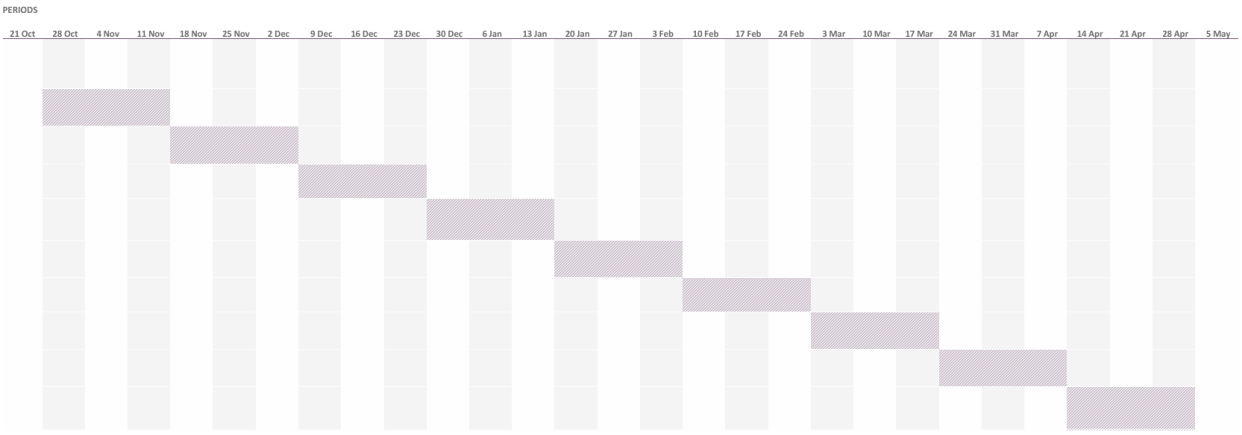
A.18 Summary

The Smart Aqua dashboard allows users to run and review the fish analysis workflow from one interface. The main workflow is to add a video, run the pipeline, review crops, inspect filtered outputs, and examine the visual clusters. The detector metrics, results graph, confusion matrix, CSV files, and output folders all support transparent review of the system. Although Smart Aqua does not replace expert judgement or provide confirmed species identification, it provides a practical tool for reducing manual review and organising fish video outputs.

Appendix B. Project Gantt chart

Project Planner

ACTIVITY	PLAN START	PLAN DURATION
Week 1–2 (14–27 Oct 2025): Project Setup & Ethics Approval — Define project scope, research goals, and data requirement. Complete ethical review and confirm dataset licensing compliance. Milestone: Project plan and ethics approved.	25 Oct 2025	3
Week 3–5 (28 Oct – 17 Nov 2025): Dataset Acquisition & Preparation — Gather open-source fish image datasets, verify data quality, clean and label images, and organize data for model training. Milestone: Dataset ready for AI model training.	28 Oct 2025	21
Week 6–8 (18 Nov – 8 Dec 2025): Baseline AI Model Development — Develop and train a baseline convolutional neural network for species recognition using cleaned datasets. Milestone: Working prototype with initial accuracy results.	##########	21
Week 9–11 (9–29 Dec 2025): Fish Health Detection Module — Enhance the AI model to detect visual indicators of fish illness and stress. Begin testing under varied lighting and image conditions. Milestone: Health detection accuracy validated on test data.	9 Dec 2025	21
Week 12–14 (30 Dec 2025 – 19 Jan 2026): Invasive Species Identification — Train and test the model to identify invasive or non-native species and compare results across datasets. Milestone: Invasive species detection functioning reliably.	30 Dec 2025	21
Week 15–17 (20 Jan – 9 Feb 2026): Population Tracking and Counting System — Implement algorithms for counting fish and detecting changes in population size across images and videos. Milestone: Population tracking system integrated.	20 Jan 2026	21
Week 18–20 (10 Feb – 2 Mar 2026): Evaluation and Optimization — Conduct performance testing, evaluate accuracy, handle bias detection, and refine model parameters. Milestone: Model performance optimized and benchmarked.	10 Feb 2026	21
Week 21–23 (3–23 Mar 2026): Dashboard and Visualization Development — Design a simple dashboard to display data on fish health, species distribution, and population trends. Milestone: Dashboard prototype complete.	3 Mar 2026	21
Week 24–26 (24 Mar – 13 Apr 2026): Integration and Final Testing — Integrate all system components, conduct user testing, and document software reliability. Milestone: Fully functional SmartAquatic system ready for final report.	##########	21
Week 27–29 (14 Apr – 5 May 2026): Final Report and Presentation — Prepare final documentation, evaluation summary, and project presentation materials. Milestone: SmartAquatic project submitted successfully.	14 Apr 2026	21



Appendix C. Agile Sprint Plan

The Smart Aqua project was managed using an iterative agile approach. The project was divided into time-boxed sprint periods, with each sprint focusing on a specific stage of research, development, testing, or reporting. The original Gantt chart helped provide an overall schedule, while the agile sprint plan allowed the project to adapt as technical challenges and dataset limitations became clearer.

Sprint	Dates	Sprint Focus	Main Tasks	Sprint Output / Milestone
Sprint 0 / Setup	14 Oct – 27 Oct 2025	Project setup and ethics preparation	Define project scope, research aims, initial dataset requirements, and ethical/data usage considerations. Begin project planning and identify possible risks.	Project idea approved, initial scope defined, and early risk/ethics considerations recorded.
Sprint 1	28 Oct – 17 Nov 2025	Dataset acquisition and preparation	Search for suitable fish footage, investigate possible aquarium data sources, gather alternative video sources, extract frames, and begin checking image quality.	Initial dataset sources identified and usable frames prepared for annotation.
Sprint 2	18 Nov – 8 Dec 2025	Baseline detector development	Create an early single-class fish dataset, begin YOLOv8 training, test whether the detector can identify fish, and inspect first results.	First working detector prototype produced. Early issues with data quality and detection performance identified.
Sprint 3	9 Dec – 29 Dec 2025	Dataset refinement and annotation improvement	Improve frame selection, remove unclear or repeated examples, annotate fish using bounding boxes, and revise the training dataset.	Improved annotated dataset prepared for further training.
Sprint 4	30 Dec – 19 Jan 2026	Detector iteration and validation	Train updated detector versions, compare dataset versions, inspect precision, recall, mAP50, and mAP50-95, and decide whether dataset quality was improving.	Stronger detector results achieved and main evaluation metrics established.
Sprint 5	20 Jan – 9 Feb 2026	Frame and crop pipeline development	Build pipeline stages for extracting frames from video, selecting useful frames, running detection, and saving fish crops.	Working video-to-crop pipeline created.
Sprint	10 Feb – 2 Mar	Crop filtering and output	Add crop quality filtering based on size, sharpness, contrast, aspect ratio,	Cleaner crop outputs produced and poor-quality

Sprint	Dates	Sprint Focus	Main Tasks	Sprint Output / Milestone
6	2026	organisation	and duplicate detection. Save filtering outputs and reports.	crops reduced before clustering.
Sprint 7	3 Mar – 23 Mar 2026	Clustering and dashboard prototype	Add ResNet50 feature extraction and HDBSCAN clustering. Begin creating the dashboard interface for running the system and opening outputs.	Filtered crops grouped into visual clusters and early dashboard prototype created.
Sprint 8	24 Mar – 13 Apr 2026	Integration and final testing	Connect the detector, frame extraction, crop extraction, filtering, clustering, and dashboard into one workflow. Test output folders, metrics, graphs, and dashboard controls.	Fully integrated Smart Aqua workflow produced and ready for evaluation.
Sprint 9	14 Apr – 5 May 2026	Final report and presentation preparation	Complete evaluation, discussion, conclusion, appendices, user manual, risk assessment, figures, and final report formatting.	Final Smart Aqua report, project evidence, and presentation materials prepared for submission.

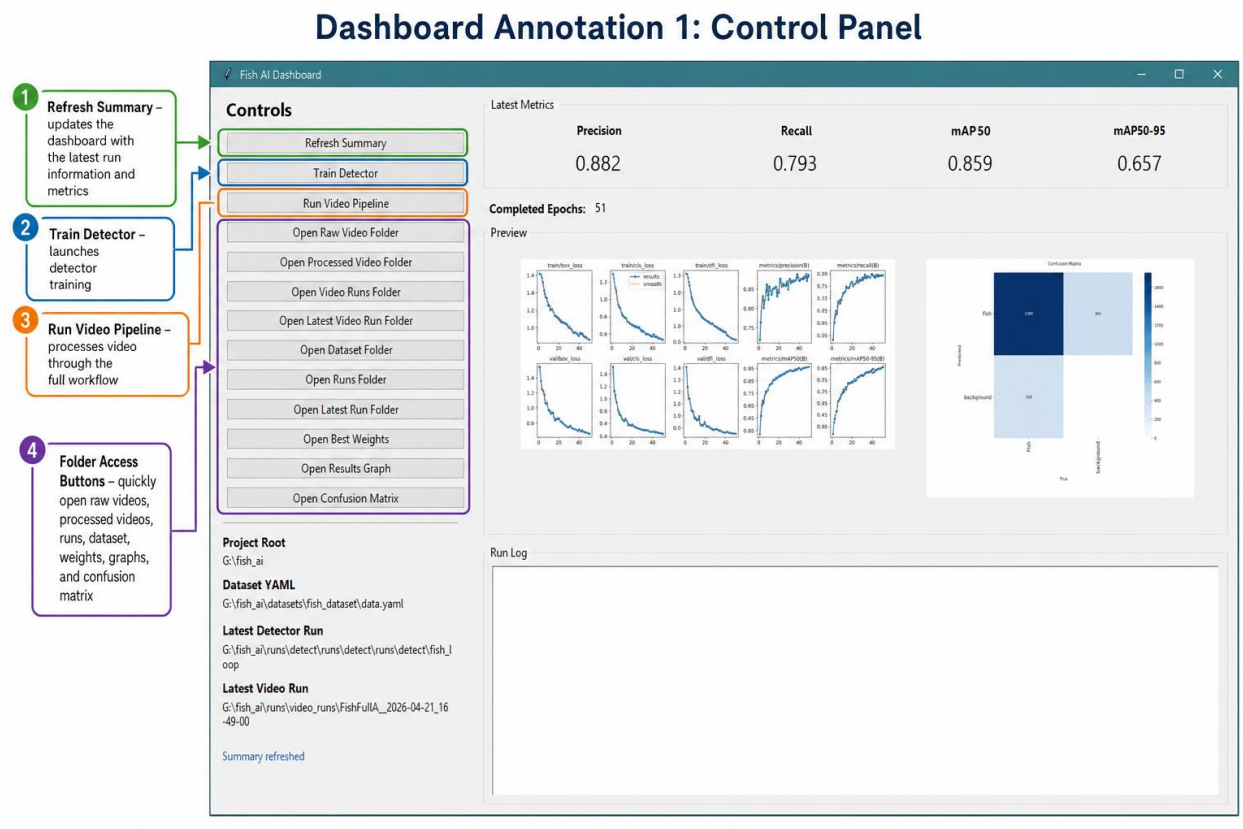
Appendix D. Risk Assessment Table

Risk ID	Risk	Likelihood	Impact	Mitigation / Response Plan
R1	Insufficient or poor-quality dataset	Medium	High	Use multiple data sources, review frames manually, remove unclear examples, and apply augmentation to improve variety.
R2	Dataset lacks visual variety	Medium	High	Include fish of different sizes, colours, positions, lighting conditions, and backgrounds to reduce bias.
R3	Original planned dataset unavailable	High	Medium	Use alternative footage sources, such as online video platforms and mobile-recorded aquarium footage.
R4	Annotation inconsistency	Medium	High	Manually review bounding boxes, use a single “Fish” class, and avoid labelling unclear or heavily obscured fish.
R5	Missed fish labels in	Medium	High	Recheck crowded frames carefully and remove

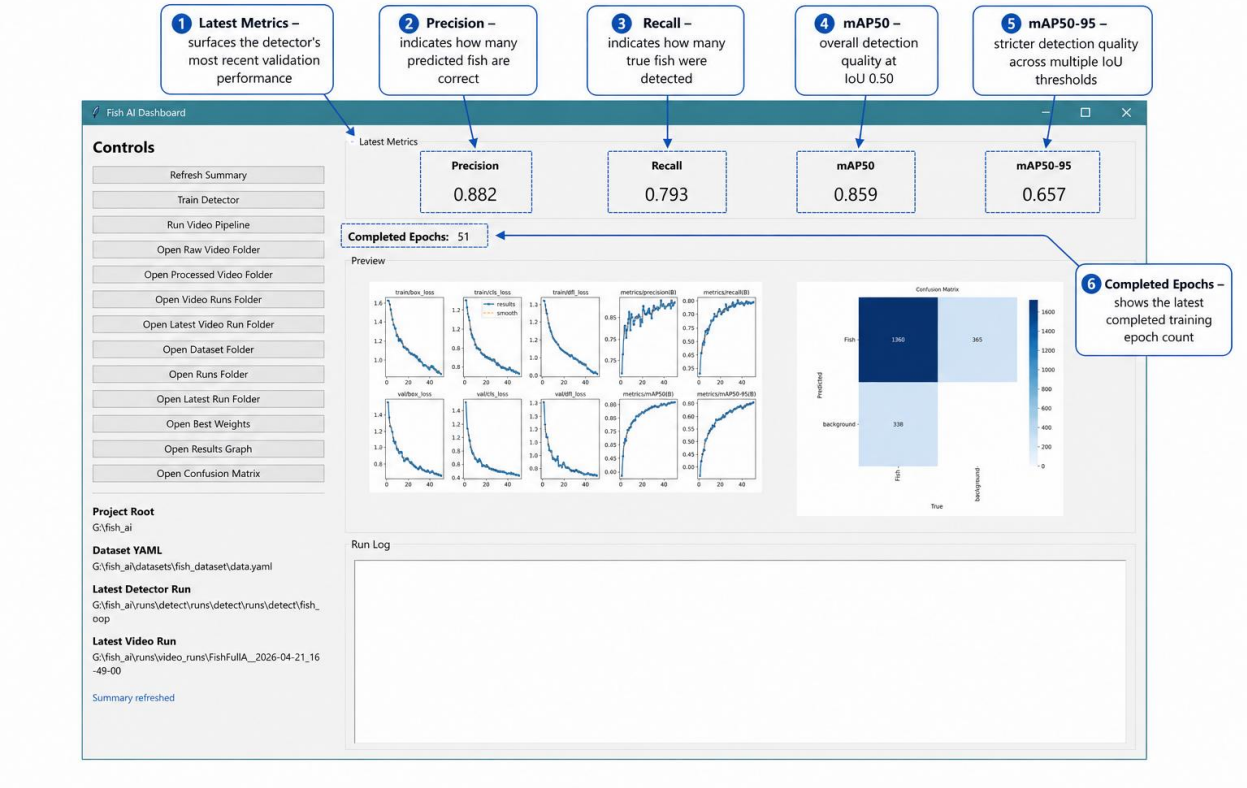
Risk ID	Risk	Likelihood	Impact	Mitigation / Response Plan
	training data			frames where fish cannot be reliably labelled.
R6	Model accuracy too low	Medium	High	Retrain using improved dataset versions, review validation metrics, and adjust training settings where practical.
R7	Low recall / missed detections	Medium	Medium	Tune inference confidence threshold, improve dataset variety, and review failure cases such as small or overlapping fish.
R8	False positives from background objects	Medium	Medium	Use crop filtering, inspect false detections, and add more varied negative/background examples where possible.
R9	Poor bounding box placement	Medium	Medium	Review annotation quality and evaluate mAP50-95 to assess stricter bounding box accuracy.
R10	CPU-only training increases experiment time	High	Medium	Keep batch size and image size manageable, train in controlled runs, and avoid unnecessary retraining once results are strong enough.
R11	Hardware or processing limitations	High	Medium	Use efficient settings, limit maximum processed frames, and keep the system lightweight for local hardware.
R12	Software errors or broken pipeline stages	Medium	High	Test each script separately, use version control, and check outputs after each stage.
R13	Dependency or library compatibility issues	Medium	Medium	Record key libraries, keep the environment consistent, and avoid unnecessary package changes late in development.
R14	Loss of files or data corruption	Low	High	Back up code, datasets, trained weights, screenshots, and report files to cloud or external storage regularly.
R15	Generated files make the project repository too large	Medium	Medium	Use .gitignore to exclude generated frames, crops, runs, builds, and temporary files from GitHub.

Risk ID	Risk	Likelihood	Impact	Mitigation / Response Plan
R16	Dashboard executable fails after being moved	Medium	Medium	Keep the EXE with its _internal folder and use a desktop shortcut rather than moving the executable alone.
R17	Pipeline outputs difficult to interpret	Medium	Medium	Save crops, filter reports, cluster summaries, metrics, graphs, and confusion matrices so outputs can be reviewed clearly.
R18	Clustering mistaken for confirmed species classification	Medium	High	Clearly state that clusters are exploratory visual groupings, not authoritative species labels.
R19	Poor crop quality affects clustering	Medium	Medium	Apply crop quality filtering based on size, sharpness, contrast, aspect ratio, and duplicate detection before clustering.
R20	Environmental factors affect image quality	High	Medium	Include varied footage and use preprocessing/augmentation to account for lighting, blur, motion, and aquarium background conditions.
R21	Time management and project delays	Medium	High	Use milestones, prioritise report writing once the system is working, and avoid excessive retraining late in the project.
R22	Loss of motivation or burnout	Medium	Medium	Break work into smaller tasks, alternate between coding and report writing, and keep a clear list of remaining priorities.
R23	Ethical or data usage concerns	Low	Medium	Use data only for educational research, avoid identifiable people, and reference dataset and tool sources appropriately.
R24	Results are overclaimed	Medium	High	Present the system as a lightweight support workflow, not a replacement for expert review or a confirmed species-identification system.
R25	Limited generalisability to new environments	Medium	High	Clearly state that more testing is needed on broader video sources, different aquariums, and varied fish populations.

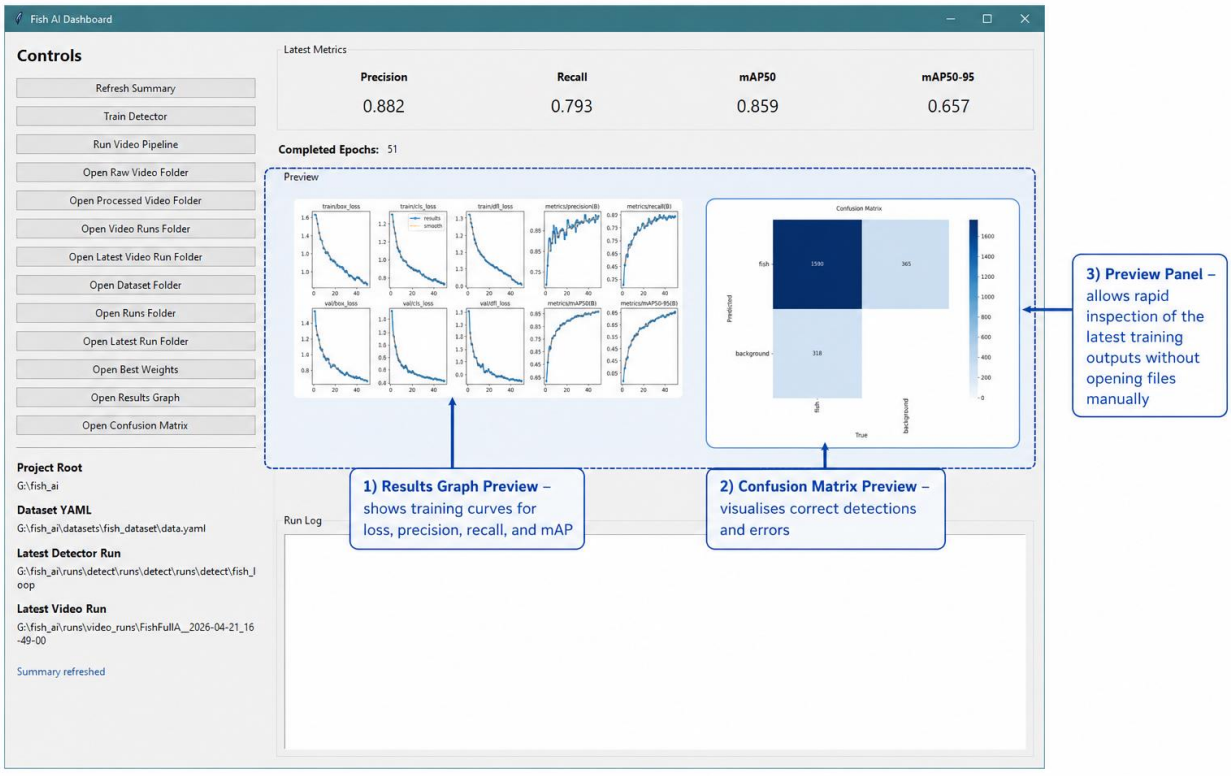
Appendix E. UI Screenshots

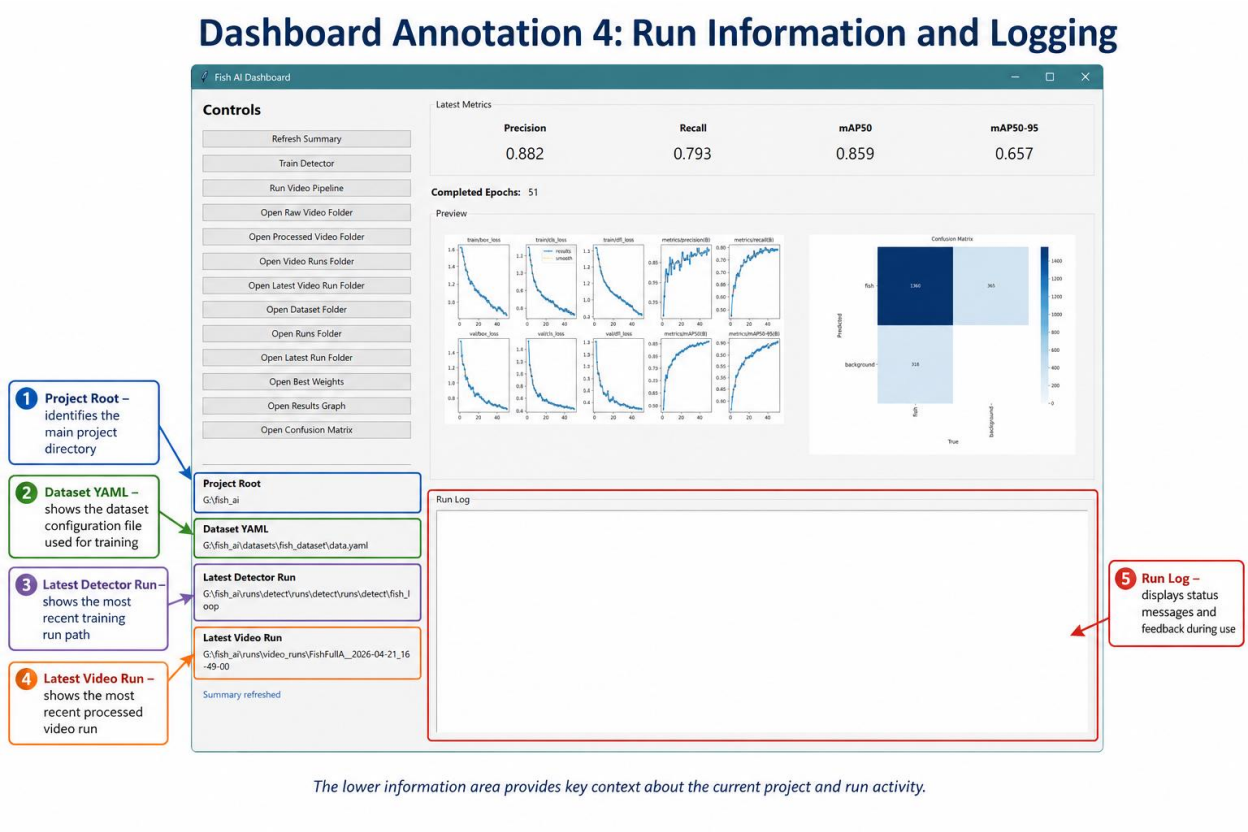


Dashboard Annotation 2: Performance Metrics



Dashboard Annotation 3: Visual Evaluation Preview





The lower information area provides key context about the current project and run activity.

Appendix F: Testing Evidence Table

Test area	What was checked	Evidence
Detector training	best.pt, metrics, results graph produced	Training run folder
Frame extraction	Video split into frames	Extracted frame folder
Crop extraction	Fish detections saved as crops	Crop output folder
Crop filtering	Poor crops removed and logged	filter_report.csv
Clustering	Crops grouped into folders	Cluster folders and CSV files
Dashboard	Buttons open correct folders/results	UI screenshots

Appendix G: Student Declaration of AI Tools

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☒
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☒
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☒
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☒
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☒
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing	☒

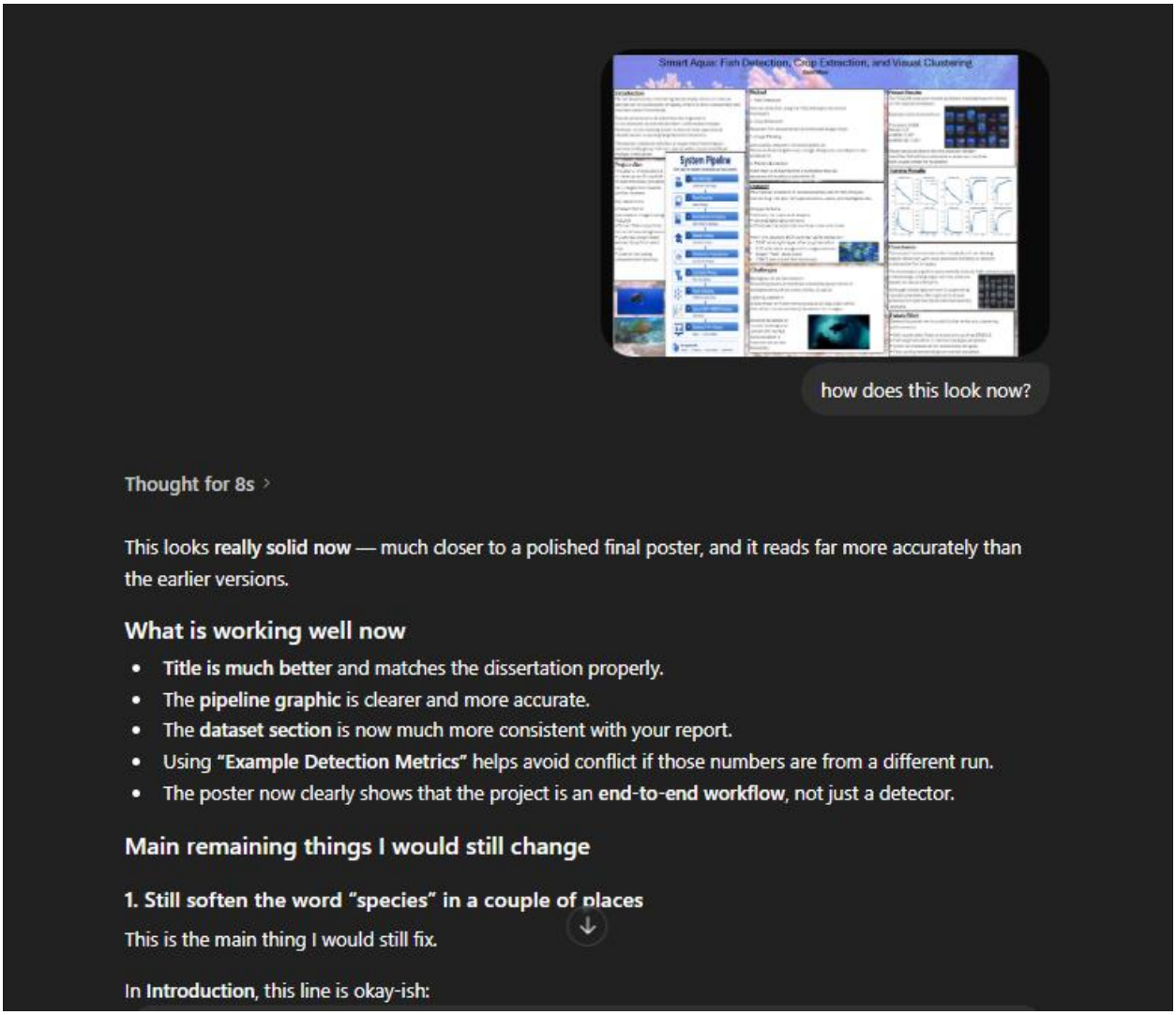
	comprehensive test plans and interpreting test results.	
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to:

Generative AI was used to assist during the development and documentation of this project. It supported planning and structuring the report, refining academic language, reviewing code for possible issues, explaining technical concepts, and helping debug problems during implementation. It was also used to suggest ways of describing the system architecture, evaluation strategy, limitations, and user manual.

AI support was used for technical guidance around the YOLOv8 detector workflow, crop extraction, crop filtering, clustering, dashboard functionality, GitHub organisation, and final report presentation. Any code, results, implementation decisions, testing, and final submitted artefacts were reviewed, adapted, and understood by me. The final project, report content, model training, evaluation evidence, and submission decisions remain my own responsibility.

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒



Above is a example of how Chat GPT was used to double check my final product and to ensure that all the work was correct while giving help for ideas on improvement